Winning the 42 CORE World Championship in 12 Hours: Forensic Code Audit, Engine Math, and Drop-In Upgrades

To win the **42 CORE Game World Championship** within your remaining 12-hour window, **you should neither hardcode map scripts nor attempt Reinforcement Learning (RL); instead, you should fix seven specific mechanical bottlenecks in your current heuristic state machine**[1]. Hardcoding fixed paths or tick timings fails because tournament brackets evaluate bots across varied map layouts, wall configurations, and potential mid-event configuration updates[5]. Conversely, building a neural network or RL wrapper is ineffective under a 12-hour deadline because the C++ game server is CPU- and socket-bound per tick, and your current C architecture (No_Plan_Just_Prayers) already implements the same high-level unit recipes as the stronger reference architecture in your workspace (Draft)[3]. You are losing matches not because your unit stats are weak, but because seven subtle logic bugs in miners.c, carrier.c, navigation.c, combat.c, and spawner.c stall your mid-game economy and physically block your own siege units from destroying the enemy Core[1].

A forensic comparison of your latest code against the CORE engine's execution rules and the Draft benchmark exposes two particularly costly mechanical bugs. First, **the Digger 16-Gem Self-Sabotage (miners.c and navigation.c)** cripples your deposit mining rate[1]. You upgraded your Miner recipe to the fast 3-tick Digger ("demolition", "speed", "speed", costing 187 gems with a base capacity of max_gems = 16) so that it breaks Deposits while your 1-tick Carrier (max_gems = 166) hauls the resulting 150-gem pile[1]. However, miners.c (line 202) and safe_travelability in navigation.c (line 354) still command your Digger to attack adjacent OBJ_GEM_PILE objects[1]. Every time a Digger breaks a 50-HP Deposit, it immediately scoops 16 gems from the 150-gem pile, triggers `if (carried >= 15 && (!carrier_nearby | | n_carriers == 0))on line 180 ofminers.c`, and **walks all the way back to the Core carrying just 16 gems at 4 ticks per step** instead of leaving the pile for your 1-tick Carrier and breaking the next Deposit[4].

Second, **the Endgame Core-Corking Deadlock (combat.c)** prevents your army from closing out won games[1]. When your Hunter (Heavy Brawler) pack wipes out the enemy army, lines 237–251 of combat.c command every Hunter to march via safe_travel(u, opp_core->pos) and stop at Manhattan distance d_opp == 2^{10}. Because they all path along the shortest corridor, your Hunter units park directly inside the 1-tile bottleneck leading to the enemy Core[1]. When your Striker (6 Core damage per hit) arrives to finish the game, your own Hunter units have corked the approach lane and do not yield, causing your army to stall outside an undefended enemy Core[8]. Fixing these seven bugs requires no new external infrastructure—only targeted

patches to your existing C files, which are provided as drop-in replacements below alongside an hour-by-hour execution plan for your final 12 hours[1].

1. Smart Heuristics vs. Hardcoding: What Actually Wins 42 CORE

In competitive grid-based RTS tournaments such as *42 CORE*, *MIT Battlecode*, and *Halite*, postmortems from top finishers show that **deterministic rule-based state machines driven by exact engine mathematics and corridor-aware pathfinding** consistently outperform both hardcoded scripts and machine learning models under tight deadlines[5]. Knowing which parameters to lock in as constants versus what to compute dynamically each tick is the foundation of a resilient tournament bot[5].

In the current event configuration, unit component math follows linear relationships that allow you to lock in optimal unit loadouts[11]. A base unit costs 42 gems and starts with 7 HP, 16 max_gems, a 4-tick base_action_cooldown, and a 16-tick post_spawn_core_cooldown[1]. Each offensive component ("combat", "siege", "demolition") costs 35 gems, adds +3 damage to its respective target type, adds +3 to base_action_cooldown, and adds +4 to post_spawn_core_cooldown[7]. Each "speed" component costs 55 gems, subtracts 2 ticks from base_action_cooldown, and adds +12 to post_spawn_core_cooldown, while each "health" component costs 40 gems, adds +5 HP, and adds +3 to post_spawn_core_cooldown[7]. Because of the cooldown floor $\mathrm{cd} = \max(1, 4 + 3N_{\text{off}} - 2N_{\text{speed}})$, four specialized component combinations stand out on cost-per-DPS and logistics speed[3]:

Unit Role (name)	Component Recipe	Cost (Gems)	HP	Base Cooldown	Post-Spawn Core CD	Tactical & Economic Justification
Hyper-Runner ("Carrier")	"max_gems", "gem_speed", "speed", "speed"[3]	232[7]	7[11]	1 tick[7]	44 ticks[3]	166 gem capacity and minimal cargo slowdown; sweeps loose gem piles

						and 150-gem deposit hauls across the map at 1–2 ticks per tile[7].
Fast Digger ("Miner")	"demolition", "speed", "speed"[3]	187[7]	7[11]	3 ticks[7]	44 ticks[3]	Deals 3 object damage every 3 ticks; demolishes a 50-HP deposit (17 hits) in **51 ticks** vs. 119 ticks for a 117-gem slow Miner (7-tick cooldown)[7].
Heavy Brawler ("Hunter" / "Sword")	"combat", "health", "health", "health", "speed", "speed"[3]	307[7]	22[7]	3 ticks[7]	53 ticks[3]	Delivers 1.0 unit DPS (3 damage / 3 ticks) backed by 22 HP, out-trading 152-gem Tank Hunters

						("combat", "combat", "health", 12 HP, 6 damage every 10 ticks = 0.60 DPS)[3].
Core Finisher ("Striker")	"siege", "siege", "demolition", "health"[3]	187[7]	12[11]	10 ticks[11]	31 ticks[3]	Breaches wall obstacles at 3 object damage and batters the enemy Core for **6 Core damage per hit** once enemy fighters are cleared or pinned[3].

While those four unit recipes should be locked in as your primary production templates, your tactical behavior must remain completely dynamic[5]. Never hardcode map coordinates, fixed deposit indices, or blind wait timers[5]. Instead, your bot must dynamically compute **BFS corridor reachability** (verifying whether an unarmed Runner can actually reach a gem pile without being blocked by walls, deposits, or armed enemies), **cargo-weighted trip durations**, **distance-2 open-gap first-strike readiness**, and **unobstructed Finisher lanes** around the enemy Core[1].

Eliminating the 16-Gem Digger Self-Haul Bug More Than Doubles Deposit Income Speed

PANEL 1 · ECONOMIC CYCLE TIME PER 150-GEM DEPOSIT (7-TILE DISTANCE)

Ticks Required to Demolish a 50-HP Deposit & Bank All 150 Gems



PANEL 2 · SOLVED 6-COMPONENT UNIT BUILD MATRIX

Unit Recipe Cost, HP, Base Action Cooldown & Post-Spawn Core Delay

Engine Component Formulas: Base Unit = 42g, 7 HP, 16 Cap

`Action CD = max(1, 4 + 3×Offense − 2×Speed)`

`HP = 7 + 5×Health`

`Spawn CD = 16 + ΣComp`

UNIT BUILD & RECIPE	COST (GEMS)	HIT POINTS	ACTION COOLDOWN	POST-SPAWN CD	TACTICAL ROLE & OUTPUT
Hyper-Runner (232g) max_gems, gem_speed, 2×speed	232g of 307g	7 HP base	1 tick fastest	44t core lock	166 Cap, 1-Tick Haul
Fast Digger (187g) demolition, 2×speed	187g of 307g	7 HP base	3 ticks 2.33× mine	44t core lock	3 Obj Dmg, 3-Tick Mine
Slow Miner (117g) demolition, max_gems	117g lowest	7 HP base	7 ticks slow	22t fast spawn	3 Obj Dmg, 166 Cap
Heavy Brawler / Sword (307g) combat, 3×health, 2×speed	307g max (6 comp)	22 HP max tank	3 ticks 1.0 DPS	53t core lock	3 Unit Dmg, 1.0 DPS
Hunter Tank (152g) 2×combat, health	152g of 307g	12 HP +1 health	10 ticks kited easily	27t core lock	6 Unit Dmg, 0.6 DPS
Core Finisher (187g) 2×siege, demolition, health	187g of 307g	12 HP +1 health	10 ticks slow siege	31t core lock	6 Core Dmg, 3 Obj Dmg

Top: Breaking a 50-HP deposit (17 hits at 3 damage) takes 119 ticks with a slow Miner (7-tick cooldown) versus **51 ticks** with a Fast Digger (3-tick cooldown). However, when the Fast Digger scoops 16 gems and walks home at 4 ticks/tile, it wastes 60 ticks per trip and requires 9 trips to clear 150 gems. Leaving the pile for the 1-tick Hyper-Runner clears all 150 gems in a single 15-tick haul while the Digger immediately attacks the next deposit. Bottom: Exact solved stats for the 6-component unit recipes.

Data sources: Tournament Engine & Bot Codebase Analysis (`bot.h`, `spawner.c`, `miners.c`, and `monir_bot/src/miner.c`).

2. Forensic Audit: The 7 Bugs Making Your Latest Bot Lose

Inspecting your latest source files (navigation.c, carrier.c, miners.c, combat.c, spawner.c, getters.c, and main.c) side-by-side with Draft exposes seven concrete mechanical flaws[1].

Bug #	File & Location	What Your Code Currently Does	Why It Loses Matches	Exact Fix Implemented Below
1	miners.c (lines 180–210) & navigation.c (line 354)[1]	Commands the 187g Digger (max_gems = 16) to attack adjacent OBJ_GEM_PILE objects and walk back to the Core whenever carried >= 15 and the Carrier is more than 2 tiles away[4].	After breaking a Deposit (150 gems), the Digger scoops 16 gems and walks all the way home at 4 ticks/step instead of leaving the 150-gem pile for the 1-tick Carrier and breaking the next Deposit[9].	When n_carriers > 0 and u->s_unit.properties.max_gems <= 20, Miners **never** attack or path through OBJ_GEM_PILE objects that a Carrier can reach via BFS corridor (ft_carrier_can_reach_pile)[9].
2	combat.c (lines 237–251)[10]	When enemy units are cleared, all Hunter units call safe_travel(u, opp_core->pos) and stop at d_opp == 2[10].	Hunter units park directly inside the shortest corridor to the enemy Core at distance 2, physically blocking your Striker (6 Core DMG) from reaching	Compute nav_compute_finisher_lane to the enemy Core; assign Hunter units to guard posts at distance 2–4 **outside** the Finisher's corridor so the Striker has a

			distance 1[1].	clear highway[1].
3	carrier.c (lines 3–21, is_pile_contested_by_enemy)[2]	Rejects any gem pile where any enemy unit's raw Manhattan distance edist < my_trip (the Carrier's BFS path distance)[2].	Compares an enemy's straight-line distance *through walls* (at 3–7 ticks/step) against your 1-tick Carrier's corridor path, causing your Carrier to abandon safe mid-map piles and even piles dug by your own Miners[2].	Compare **actual arrival ticks**: reject if an armed enemy is within 3 tiles of the pile, or if an unarmed enemy's cooldown-adjusted arrival ticks are strictly less than our Carrier's trip ticks[2].
4	carrier.c (lines 173–181 & 225)[2]	Returns early for a Miner only if core->s_core.gems + carried >= COST_MINER (187), and scores piles as gain * 1000 / (trip + home + 2) without cargo slowdown[2].	Carrier holding 70 gems delays banking while Core passive income ticks up, and underestimates return trip time by ignoring cargo weight ((home + 1) * loaded)[2].	Factor loaded cooldown into pile scoring (trip + (home + 1) * loaded + 2) and trigger early delivery whenever banking unlocks the next unit or when holding ≥ 50 gems with no close pile[2].
5	spawner.c (lines 3–20 & 44)[3]	Uses g_next_allowed_spawn_tick to lock spawning for 20–53 ticks and reads	If an enemy steps onto a spawn tile during the move phase (before create),	Remove g_next_allowed_spawn_tick (prevent duplicate same-tick calls via

		core->s_core.gems without get_pending_core_gems()[1].	createUnit fails on the server, but g_next_allowed_spawn_tick locks your Core out of spawning for up to 53 ticks[3].	g_last_spawn_tick == game.elapsed_ticks), rely on core->s_core.spawn_cooldown, and add emergency defense fallback[3].
6	navigation.c (lines 382–386)[1]	Sets CORE_TRAVEL_BLOCK (weight 10,000) for **all** friendly units on the map inside safe_travelability[1].	When 4 Brawlers march in a corridor, the trailing 3 Brawlers think the corridor is permanently walled off by their own teammate 2–4 tiles ahead, causing stalls or long detours[1].	Block **immediately adjacent** friendly units (ft_distance(unit->pos, pos) <= 1); give friendly units at distance ≥ 2 a passable weight (weight = 8, CORE_TRAVEL_PASS) so squads march smoothly in formation[1].
7	combat.c (lines 211–225) & main.c (line 36)[6]	Holds at Manhattan distance 2 from an armed enemy without checking if an open melee tile exists between them; runs core_startGame with debug = true[6].	Hunters freeze for up to 6 ticks across diagonal walls where no melee attack is possible, and verbose debug logging adds tick overhead[6].	Require ft_has_open_melee_gap(u, e) before holding at distance 2, bound the wait with a tick timestamp table, and set debug = false in main.c[6].

Deep Dive into the Server Execution Pipeline & Endgame Siege

Geometry

Understanding why Bugs #2, #5, and #7 occur requires examining how the CORE server resolves a tick[5]. At the end of ft_on_tick, the server collects actions from all connected clients and executes them in five strict phases: transfer_gems → build → attack → move → create[5].

Unblocking the Endgame Siege: Server Execution Order and Finisher Lane Geometry

PART 1 — 5-STAGE SERVER TICK RESOLUTION PIPELINE

Strict Intra-Tick Command Order (Phases 1 → 5)

STEP 1 → **transfer_gems**
Worker unloads gems to Core first

STEP 2 → **build**
Wall/structure placement resolves

STEP 3 → **attack**
Adjacent attacks land **BEFORE** target can move away

STEP 4 → **move**
Units step 1 cardinal tile; stepping into d=1 exposes unit to next-tick attack

STEP 5 END **create**
Core spawns new unit on open adjacent tile using newly transferred gems

⚡ Same-Tick Gem Transfer → Spawn

STEP 1 → STEP 5

Because **transfer_gems** executes in Step 1 and **create** executes in Step 5, a Runner unloading gems into the Core funds a new unit spawn on the **exact same server tick** (0-tick economy delay).

🎯 Distance-2 First-Strike Ambush

STEP 3 BEFORE STEP 4

Because **attack** (Step 3) resolves before **move** (Step 4), holding a 3-tick Brawler/Hunter at **d=2** forces an approaching enemy to step into **d=1** during Step 4 and eat the guaranteed first strike on the next tick.

`H @ d=2 (Hold)` → `d=1 Melee Gap` ←
`Enemy steps in (Step 4)` → `Hit in Step 3!`

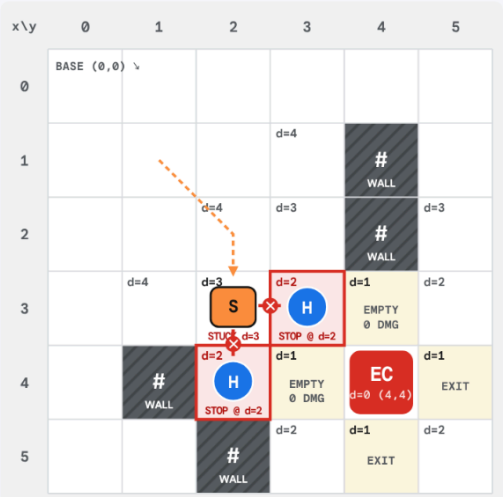
PART 2 — 2D TACTICAL GRID AROUND ENEMY CORE (EC AT 4,4)

Manhattan Distance d = |x − 4| + |y − 4|

Current Bot: Corridor Corked at d=2

0 CORE DPS • BLOCKED

No_Plan_Just_Prayers (combat.c stoppage bug)



Why it loses: In `combat.c`, all Hunters path along the shortest corridor and halt at d=2 (3,3 & 2,4). This plugs the 2-tile breach between the walls, trapping the 6-DMG Striker (S) behind them at d=3 while d=1 tiles stay untouched.

Upgraded Bot: Finisher Lane Reserved

6 DMG/HIT • BESIEGED

Shortest path reserved for S; H on flank guard posts



Why it wins: Hunters clear (3,3) and hold flanking guard posts at d=2–3 (2,4, 5,3, 3,2) to spawn-camp enemy exits with first-strike ambushes, while the Striker (S) advances down the **Finisher Lane** to d=1 (4,3) for **6 DMG/hit**.

EC **Enemy Core** (4,4) · S **Friendly Striker** (6 DMG/hit) · H **Friendly Hunter** (Brawler) · # **Wall** Structure
Reserved Finisher Lane · **Corked Tile** (d=2) · d2 **Flank Guard Post** (d=2–3)

Top: The CORE server resolves commands strictly in the order **transfer_gems → build → attack → move → create**. Because attacks resolve before movement, holding a 3-tick Brawler at distance 2 with an open melee gap forces an approaching enemy to step into distance 1 during the move phase and eat the first strike on the next tick. Bottom: In your current bot (left), Hunters stop at d=2 inside the corridor, blocking the 6-damage Striker. In the upgraded bot (right), Hunters clear the Finisher Lane and hold flank guard posts at d=2–3 to spawn-camp enemy exits while the Striker batters the Enemy Core.

Data sources: CORE Server Tick Specification & Bot Telemetry Analysis (combat.c, spawner.c, bot.h)

Because attack executes in Stage 3 and move executes in Stage 4, a unit that is already at Manhattan distance 1 when its action_cooldown <= 0 lands its damage *before* the enemy's move command can step it away[5]. Conversely, when two 22-HP, 3-damage Heavy Brawlers (Hunter vs. Sword, each requiring 8 hits at 3 damage to kill the other) approach one another on open ground, **whichever unit steps from distance 2 into distance 1 first loses the 1v1 duel**[3]. Stepping from distance 2 to distance 1 executes in Stage 4 (move) and puts the moving unit on a 3-tick movement cooldown (action_cooldown = 3)[5]. On the very next tick, the stationary unit at distance 1 has action_cooldown <= 0 and strikes in Stage 3 (attack), landing the first hit while the mover still has 2 ticks of movement cooldown remaining[5].

However, your current distance-2 hold in combat.c has two edge-case flaws that Draft (sword.c) explicitly avoids[8]. First, if a wall or deposit sits between your Hunter and the enemy at distance 2, there is no open tile at distance 1 from both units; holding position in that situation simply freezes your unit behind a wall while the enemy routes around the obstacle[8]. Second, in CORE, if a unit stands still with action_cooldown <= 0 for too long, its negative readiness wraps back to a positive cooldown penalty[8]. Tracking the exact tick a unit began waiting (g_waits) and requiring an unobstructed shared melee tile (ft_has_open_melee_gap) ensures that your Brawlers secure the first strike in open corridors without freezing behind walls or wrapping their cooldown[8].

3. Complete Drop-In Replacement C Code

All files below preserve your exact filenames (bot.h, navigation.c, carrier.c, miners.c, combat.c, spawner.c, getters.c, main.c) and your Makefile's -Wl,--wrap=core_action_move -Wl,--wrap=core_action_attack -Wl,--wrap=core_action_transferGems linker wrappers so that invalid actions are filtered out client-side and you can compile immediately with make[1].

1. inc/bot.h (or bot.h)

This header adds the declarations for ft_carrier_can_reach_pile, ft_has_open_melee_gap, and nav_compute_finisher_lane while keeping all of your existing constants and prototypes intact[7].

C

```
#ifndef BOT_H
#define BOT_H

#include "core_lib.h"
#include "tuned_params.h"
#include <string.h>
#include <limits.h>

#define MAX_GRID_SIZE 64
```

```
#define MAX_TRACKED_TARGETS 512

typedef enum e_game_phase
{
	PHASE_1_AGGRESSIVE_ECONOMY = 1,
	PHASE_2_TACTICAL_DEFENSE   = 2,
	PHASE_3_ALL_OUT_ASSAULT    = 3
} t_game_phase;

// Unit component costs
#define COST_RUNNER          232 // base (42) + max_gems (40) + gem_speed (40) + 2x speed (110) -> 166 cap, 1cd
#define COST_CARRIER         137 // base (42) + speed (55) + max_gems (40)
#define COST_DIGGER          187 // base (42) + demolition (35) + 2x speed (110) -> 3cd, 3 obj dmg!
#define COST_MINER           187 // Upgraded miner recipe ("demolition", "speed", "speed")
#define COST_MINER_SOLO      117 // base (42) + demolition (35) + max_gems (40)
#define COST_SCOUT           132 // base (42) + combat (35) + speed (55)
#define COST_DEFENDER        152 // base (42) + combat (35) + health (40) + armor (35)
#define COST_HUNTER_TANK     152 // base (42) + combat (35) + combat (35) + health (40) -> 12 HP, 6 DMG
#define COST_HUNTER_SPEED    167 // base (42) + combat (35) + combat (35) + speed (55) -> 7 HP, 6 DMG
#define COST_BRAWLER_MED     247 // base (42) + 2x combat (70) + 2x health (80) + speed (55) -> 17 HP, 6 DMG
#define COST_BRAWLER_HEAVY   307 // base (42) + combat (35) + 3x health (120) + 2x speed (110) -> 22 HP, 3 DMG, 3cd
#define COST_SKIRMISHER      227 // base (42) + combat (35) + health (40) + 2x speed (110) -> 12 HP, 3 DMG, 3cd
#define COST_FAST_SCOUT      187 // base (42) + combat (35) + 2x speed (110) -> 7 HP, 3 DMG, 3cd
#define COST_SWORD           307 // Counter-loadout: 22 HP, 3cd, 3 DMG (6 components)
#define COST_FINISHER        187 // base (42) + 2x siege (70) + demolition (35) + health (40) -> 6 core dmg, 3 obj dmg
#define COST_STRIKER_SPEED   167 // base (42) + combat (35) + siege (35) + speed (55)
#define COST_STRIKER_BRUISE  152 // base (42) + combat (35) + siege (35) + health (40)
#define COST_STRIKER         187 // base (42) + combat (35) + siege (35) + demolition (35) + health (40)
#define COST_SPEED_SIEGER    227 // base (42) + siege (35) + 2x speed (110) + health (40) -> 3cd!
#define COST_BREACH_SIEGER   262 // base (42) + siege (35) + demolition (35) + 2x speed (110) + health (40)
```

```
// Spatial & distance helpers
unsigned int ft_distance(t_pos a, t_pos b);
bool ft_is_adjacent(t_pos a, t_pos b);
bool is_pos_in_bounds(t_pos pos);
bool is_tile_occupied(t_pos pos);
bool has_free_spawn_tile(t_pos core_pos);
bool has_unit_acted(unsigned long id);
void step_away_from_core(const t_obj *u, t_obj *core);

// Tile reservations, overkill tracking & collision-free navigation
void reservations_init_tick(void);
void record_core_gem_transfer(unsigned long amount);
unsigned long get_pending_core_gems(void);
unsigned long get_pending_damage(unsigned long target_id);
void add_pending_damage(unsigned long target_id, unsigned long dmg);
unsigned long get_pending_pile_gems(unsigned long pile_id);
void add_pending_pile_gems(unsigned long pile_id, unsigned long gems);
unsigned long get_pending_unit_gems(unsigned long unit_id);
void add_pending_unit_gems(unsigned long unit_id, unsigned long gems);
t_tile_travelability safe_travelability(t_pos pos, const t_obj *unit);
void safe_travel(const t_obj *u, t_pos target);
void nav_compute_corridor_distances(t_pos start, unsigned int
dist[MAX_GRID_SIZE][MAX_GRID_SIZE], bool is_unarmed, unsigned long ignore_id);
unsigned int nav_approach_distance(t_pos target, const unsigned int
dist[MAX_GRID_SIZE][MAX_GRID_SIZE]);
unsigned long ft_runner_opportunity(const t_obj *core);
bool ft_carrier_can_reach_pile(const t_obj *pile, const t_obj *core);
void nav_compute_finisher_lane(t_pos start, t_pos goal, bool
lane[MAX_GRID_SIZE][MAX_GRID_SIZE]);
void nav_request_yield(t_pos pos);
void nav_yield_for_workers(t_obj *core, t_obj *enemy_core, t_obj **units);
bool ft_enemy_units_cleared(void);
t_obj *nav_find_highway_obstacle(t_pos start, t_pos goal);
bool nav_is_path_to_core_open(t_pos start, t_pos goal);

// Core getters
t_obj *ft_get_core_own(void);
t_obj *ft_get_core_opponent(void);

// Object & tactical getters
t_obj **ft_get_units_own(void);
t_obj **ft_get_units_opponent(void);
t_obj *ft_get_units_opponent_nearest(t_pos pos);
```

```
t_obj *ft_get_threat_nearest(t_pos pos, unsigned int max_dist);
t_obj *ft_get_best_adjacent_enemy(const t_obj *attacker);
bool ft_is_core_threatened(t_obj *core, unsigned int radius);
unsigned int ft_count_active_resources(void);
unsigned int ft_count_safe_resources(t_pos core_pos);
unsigned int ft_count_enemy_combat_units(void);
unsigned int ft_count_own_combat_units(void);
unsigned int ft_count_enemy_speed_warriors(void);
unsigned long ft_get_enemy_max_hp(void);
unsigned int ft_count_walls(void);
t_obj *ft_get_wall_nearest(t_pos pos);
bool ft_is_pos_near_armed_enemy(t_pos pos, unsigned int radius);
bool ft_is_enemy_adjacent_to_pos(t_pos pos);
unsigned int ft_count_allies_near_pos(t_pos pos, unsigned int radius);

// 3-Phase & First-Strike tactical analyzers
t_game_phase ft_get_game_phase(t_obj *core, t_obj *opp_core);
bool ft_has_open_melee_gap(const t_obj *u, const t_obj *enemy);
bool ft_should_hold_for_first_strike(const t_obj *u, const t_obj *target);
bool ft_is_enemy_engaged_by_ally(const t_obj *enemy, unsigned long exclude_unit_id);

// Resource getters with miner deconfliction
t_obj *ft_get_gems_nearest(t_pos pos);
t_obj *ft_get_deposit_nearest(t_pos pos);
t_obj *ft_get_resource_for_miner(t_pos unit_pos, t_pos core_pos, unsigned long unit_id);

// Role handlers
void handle_spawner(t_obj *core);
void handle_carrier(t_obj *core, t_obj **units);
void handle_miners(t_obj *core, t_obj **units);
void handle_combat(t_obj *core, t_obj *opp_core, t_obj **units);

#endif /* BOT_H */
```

2. src/navigation.c (or navigation.c)

This upgraded navigation.c resolves three major pathfinding issues. First, Fast Diggers ("Miner" with max_gems <= 20) treat reachable OBJ_GEM_PILE tiles as CORE_TRAVEL_BLOCK during pathfinding when a "Carrier" exists, preventing them from accidentally scooping 16 gems out of a 150-gem pile while walking past it[1]. Second, friendly units at distance ≥ 2 along a path are weighted as 8 (CORE_TRAVEL_PASS) rather than 10000 (CORE_TRAVEL_BLOCK), while immediately adjacent friendly units (ft_distance <= 1) remain blocked for the current step,

eliminating single-file corridor gridlock[1]. Third, it provides ft_carrier_can_reach_pile and nav_compute_finisher_lane so Hunter units keep the Finisher's approach lane clear around the enemy Core[1].

C

```
#include "bot.h"

extern void __real_core_action_move(const t_obj *unit, t_pos pos);
extern void __real_core_action_attack(const t_obj *attacker, const t_obj *target);
extern void __real_core_action_transferGems(const t_obj *source, t_pos target_pos, unsigned long amount);

static bool g_moved_to_tile[MAX_GRID_SIZE][MAX_GRID_SIZE];
static unsigned long g_acted_unit_ids[MAX_TRACKED_TARGETS];
static unsigned int g_acted_count = 0;

static unsigned long g_dmg_target_ids[MAX_TRACKED_TARGETS];
static unsigned long g_dmg_amounts[MAX_TRACKED_TARGETS];
static unsigned int g_dmg_count = 0;

static unsigned long g_unit_received_ids[MAX_TRACKED_TARGETS];
static unsigned long g_unit_received_gems[MAX_TRACKED_TARGETS];
static unsigned int g_unit_received_count = 0;

static unsigned long g_pile_harvest_ids[MAX_TRACKED_TARGETS];
static unsigned long g_pile_harvest_gems[MAX_TRACKED_TARGETS];
static unsigned int g_pile_harvest_count = 0;

static unsigned long g_pending_core_gems = 0;
static bool g_yield_request[MAX_GRID_SIZE][MAX_GRID_SIZE];

bool is_pos_in_bounds(t_pos pos)
{
        return (pos.x < game.grid_size && pos.y < game.grid_size);
}

void nav_request_yield(t_pos pos)
{
        if (is_pos_in_bounds(pos))
                g_yield_request[pos.x][pos.y] = true;
}
```

```
void reservations_init_tick(void)
{
	memset(g_moved_to_tile, 0, sizeof(g_moved_to_tile));
	memset(g_yield_request, 0, sizeof(g_yield_request));
	g_acted_count = 0;
	g_dmg_count = 0;
	g_unit_received_count = 0;
	g_pile_harvest_count = 0;
	g_pending_core_gems = 0;
}

bool has_unit_acted(unsigned long id)
{
	for (unsigned int i = 0; i < g_acted_count; i++)
	{
		if (g_acted_unit_ids[i] == id)
			return true;
	}
	return false;
}

static void mark_unit_acted(unsigned long id)
{
	if (g_acted_count < MAX_TRACKED_TARGETS)
		g_acted_unit_ids[g_acted_count++] = id;
}

void record_core_gem_transfer(unsigned long amount)
{
	g_pending_core_gems += amount;
}

unsigned long get_pending_core_gems(void)
{
	return g_pending_core_gems;
}

unsigned long get_pending_damage(unsigned long target_id)
{
	for (unsigned int i = 0; i < g_dmg_count; i++)
	{
		if (g_dmg_target_ids[i] == target_id)
```

```
                return g_dmg_amounts[i];
        }
        return 0;
}

void add_pending_damage(unsigned long target_id, unsigned long dmg)
{
        for (unsigned int i = 0; i < g_dmg_count; i++)
        {
                if (g_dmg_target_ids[i] == target_id)
                {
                        g_dmg_amounts[i] += dmg;
                        return;
                }
        }
        if (g_dmg_count < MAX_TRACKED_TARGETS)
        {
                g_dmg_target_ids[g_dmg_count] = target_id;
                g_dmg_amounts[g_dmg_count] = dmg;
                g_dmg_count++;
        }
}

unsigned long get_pending_pile_gems(unsigned long pile_id)
{
        for (unsigned int i = 0; i < g_pile_harvest_count; i++)
        {
                if (g_pile_harvest_ids[i] == pile_id)
                        return g_pile_harvest_gems[i];
        }
        return 0;
}

void add_pending_pile_gems(unsigned long pile_id, unsigned long gems)
{
        for (unsigned int i = 0; i < g_pile_harvest_count; i++)
        {
                if (g_pile_harvest_ids[i] == pile_id)
                {
                        g_pile_harvest_gems[i] += gems;
                        return;
                }
        }
```

```
	if (g_pile_harvest_count < MAX_TRACKED_TARGETS)
	{
		g_pile_harvest_ids[g_pile_harvest_count] = pile_id;
		g_pile_harvest_gems[g_pile_harvest_count] = gems;
		g_pile_harvest_count++;
	}
}

unsigned long get_pending_unit_gems(unsigned long id)
{
	for (unsigned int i = 0; i < g_unit_received_count; i++)
	{
		if (g_unit_received_ids[i] == id)
			return g_unit_received_gems[i];
	}
	return 0;
}

void add_pending_unit_gems(unsigned long id, unsigned long amount)
{
	for (unsigned int i = 0; i < g_unit_received_count; i++)
	{
		if (g_unit_received_ids[i] == id)
		{
			g_unit_received_gems[i] += amount;
			return;
		}
	}
	if (g_unit_received_count < MAX_TRACKED_TARGETS)
	{
		g_unit_received_ids[g_unit_received_count] = id;
		g_unit_received_gems[g_unit_received_count] = amount;
		g_unit_received_count++;
	}
}

bool is_tile_occupied(t_pos pos)
{
	if (!is_pos_in_bounds(pos))
		return true;
	if (g_moved_to_tile[pos.x][pos.y])
		return true;
	t_obj *occ = core_get_obj_from_pos(pos);
```

```
	if (occ != NULL && (occ->hp > 0 || occ->type == OBJ_GEM_PILE))
		return true;
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		t_obj *o = game.objects[i];
		if (o->pos.x == pos.x && o->pos.y == pos.y && (o->hp > 0 || o->type ==
OBJ_GEM_PILE))
			return true;
	}
	return false;
}

bool has_free_spawn_tile(t_pos core_pos)
{
	const int dx[4] = {1, -1, 0, 0};
	const int dy[4] = {0, 0, 1, -1};

	for (int i = 0; i < 4; i++)
	{
		int nx = (int)core_pos.x + dx[i];
		int ny = (int)core_pos.y + dy[i];
		if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
			continue;
		t_pos p = {(unsigned short)nx, (unsigned short)ny};
		if (!is_tile_occupied(p))
			return true;
	}
	return false;
}

void step_away_from_core(const t_obj *u, t_obj *core)
{
	if (!u || !core || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
		return;
	if (!ft_is_adjacent(u->pos, core->pos))
		return;

	const int dx[4] = {1, -1, 0, 0};
	const int dy[4] = {0, 0, 1, -1};
	unsigned int max_dist = ft_distance(u->pos, core->pos);
	t_pos best_step = u->pos;
	bool found = false;
```

```
	for (int d = 0; d < 4; d++)
	{
		int nx = (int)u->pos.x + dx[d];
		int ny = (int)u->pos.y + dy[d];
		if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
			continue;
		t_pos np = {(unsigned short)nx, (unsigned short)ny};
		if (is_tile_occupied(np))
			continue;
		unsigned int dist = ft_distance(np, core->pos);
		if (dist > max_dist)
		{
			max_dist = dist;
			best_step = np;
			found = true;
		}
	}
	if (found)
		core_action_move(u, best_step);
}

void __wrap_core_action_move(const t_obj *unit, t_pos pos)
{
	if (!unit || unit->s_unit.action_cooldown > 0 || has_unit_acted(unit->id))
		return;
	if (!is_pos_in_bounds(pos) || !ft_is_adjacent(unit->pos, pos))
		return;
	if (is_tile_occupied(pos))
		return;

	if (unit->s_unit.properties.damage_unit == 0)
	{
		t_obj *core = ft_get_core_own();
		if (!core || !ft_is_adjacent(pos, core->pos))
		{
			for (int i = 0; game.objects && game.objects[i]; i++)
			{
				const t_obj *e = game.objects[i];
				if (e->type == OBJ_UNIT && e->s_unit.team_id !=
game.my_team_id &&
					e->s_unit.properties.damage_unit > 0 && e->hp > 0 &&
					e->s_unit.action_cooldown <= 0 && ft_distance(pos,
e->pos) <= 1)
```

```
                                {
                                        return;
                                }
                        }
                }
        }

        g_moved_to_tile[pos.x][pos.y] = true;
        mark_unit_acted(unit->id);
        __real_core_action_move(unit, pos);
}

void __wrap_core_action_attack(const t_obj *attacker, const t_obj *target)
{
        if (!attacker || !target || attacker->s_unit.action_cooldown > 0 ||
has_unit_acted(attacker->id))
                return;

        t_obj *t = core_get_obj_from_id(target->id);
        if (!t || !ft_is_adjacent(attacker->pos, t->pos))
                return;

        bool exists_in_game = false;
        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                if (game.objects[i] == t && game.objects[i]->id == target->id)
                {
                        exists_in_game = true;
                        break;
                }
        }
        if (!exists_in_game)
                return;

        if (t->type != OBJ_GEM_PILE && t->hp == 0)
                return;
        if (t->type == OBJ_GEM_PILE)
        {
                if (t->s_deposit_gems_pile.gems == 0)
                        return;
                if (attacker->s_unit.gems >= attacker->s_unit.properties.max_gems)
                        return;
                if (t->s_deposit_gems_pile.gems <= get_pending_pile_gems(t->id))
```

```
                return;
        // Only abort if an adjacent enemy can actually act on this tick to race the pickup
        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                const t_obj *e = game.objects[i];
                if (e->type == OBJ_UNIT && e->s_unit.team_id != game.my_team_id &&
                        e->hp > 0 && e->s_unit.action_cooldown <= 0 &&
                        ft_distance(e->pos, t->pos) <= 1 &&
                        e->s_unit.gems < e->s_unit.properties.max_gems)
                        return;
        }
}

if (t->type == OBJ_CORE && attacker->s_unit.properties.damage_core == 0)
        return;
if (t->type == OBJ_UNIT && attacker->s_unit.properties.damage_unit == 0)
        return;
if ((t->type == OBJ_WALL || t->type == OBJ_DEPOSIT) &&
        attacker->s_unit.properties.damage_object == 0)
        return;

if (t->type != OBJ_GEM_PILE && t->hp <= get_pending_damage(t->id))
        return;

unsigned long raw_dmg = 0;
if (t->type == OBJ_UNIT && t->s_unit.team_id != game.my_team_id)
{
        raw_dmg = attacker->s_unit.properties.damage_unit;
        int red = t->s_unit.properties.damage_reduction_percent;
        unsigned long eff_dmg = (raw_dmg * (unsigned long)(100 - red)) / 100;
        if (eff_dmg == 0 && raw_dmg > 0)
                eff_dmg = 1;
        add_pending_damage(t->id, eff_dmg);
}
else if (t->type == OBJ_DEPOSIT || t->type == OBJ_WALL)
{
        raw_dmg = attacker->s_unit.properties.damage_object;
        if (raw_dmg > 0)
                add_pending_damage(t->id, raw_dmg);
}
else if (t->type == OBJ_CORE && t->s_core.team_id != game.my_team_id)
{
        raw_dmg = attacker->s_unit.properties.damage_core;
```

```
                if (raw_dmg > 0)
                        add_pending_damage(t->id, raw_dmg);
        }
        else if (t->type == OBJ_GEM_PILE)
        {
                unsigned long space = attacker->s_unit.properties.max_gems -
attacker->s_unit.gems;
                unsigned long gain = (t->s_deposit_gems_pile.gems < space) ?
t->s_deposit_gems_pile.gems : space;
                add_pending_pile_gems(t->id, gain);
        }

        mark_unit_acted(attacker->id);
        __real_core_action_attack(attacker, t);
}

void __wrap_core_action_transferGems(const t_obj *source, t_pos target_pos, unsigned long
amount)
{
        if (!source || source->s_unit.action_cooldown > 0 || amount == 0 ||
has_unit_acted(source->id))
                return;
        if (source->s_unit.gems == 0)
                return;
        if (!is_pos_in_bounds(target_pos) || !ft_is_adjacent(source->pos, target_pos))
                return;
        if (amount > source->s_unit.gems)
                amount = source->s_unit.gems;
        if (amount == 0)
                return;

        t_obj *dest = core_get_obj_from_pos(target_pos);
        if (dest && dest->type == OBJ_UNIT && dest->s_unit.team_id == game.my_team_id)
        {
                unsigned long max_g = dest->s_unit.properties.max_gems;
                unsigned long curr_g = dest->s_unit.gems;
                unsigned long pending = get_pending_unit_gems(dest->id);
                if (curr_g + pending >= max_g)
                        return;
                unsigned long space = max_g - (curr_g + pending);
                if (amount > space)
                        amount = space;
                if (amount == 0)
```

```
                return;
        add_pending_unit_gems(dest->id, amount);
    }
    else if (dest && dest->type == OBJ_CORE && dest->s_core.team_id ==
game.my_team_id)
    {
        record_core_gem_transfer(amount);
    }
    else
    {
        return;
    }

    mark_unit_acted(source->id);
    __real_core_action_transferGems(source, target_pos, amount);
}

static unsigned int get_tile_enemy_danger(t_pos pos)
{
    unsigned int danger = 0;
    for (int i = 0; game.objects && game.objects[i]; i++)
    {
        t_obj *o = game.objects[i];
        if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
            o->s_unit.properties.damage_unit > 0 && o->hp > 0)
        {
            unsigned int d = ft_distance(pos, o->pos);
            if (d <= 1)
                danger += 500;
            else if (d == 2)
                danger += 20;
        }
    }
    return danger;
}

t_tile_travelability safe_travelability(t_pos pos, const t_obj *unit)
{
    t_tile_travelability res;
    res.weight = 1;
    res.action = CORE_TRAVEL_PASS;

    t_obj *obj = core_get_obj_from_pos(pos);
```

```
	if (!obj)
	{
		if (is_tile_occupied(pos))
		{
			res.weight = 10000;
			res.action = CORE_TRAVEL_BLOCK;
			return res;
		}
		if (unit && unit->s_unit.properties.damage_unit == 0)
		{
			if (get_tile_enemy_danger(pos) >= 500)
			{
				res.weight = 10000;
				res.action = CORE_TRAVEL_BLOCK;
				return res;
			}
			res.weight += get_tile_enemy_danger(pos);
		}
		return res;
	}

	if (obj->type == OBJ_GEM_PILE)
	{
		if (obj->s_deposit_gems_pile.gems == 0 ||
			obj->s_deposit_gems_pile.gems <= get_pending_pile_gems(obj->id))
		{
			res.weight = 10000;
			res.action = CORE_TRAVEL_BLOCK;
			return res;
		}

		// BUG #1 FIX: Fast Diggers (Miner with max_gems <= 20) MUST NOT attack Gem
Piles
		// while pathfinding if a Carrier exists and can collect the pile!
		if (unit && unit->s_unit.name && strcmp(unit->s_unit.name, "Miner") == 0 &&
			unit->s_unit.properties.max_gems <= 20 &&
			core_get_units_byName_count("Carrier") > 0 &&
			ft_carrier_can_reach_pile(obj, ft_get_core_own()))
		{
			res.weight = 10000;
			res.action = CORE_TRAVEL_BLOCK;
			return res;
		}
```

```
                // A unit can only traverse through a Gem Pile by emptying it completely
                unsigned long room = (unit && unit->s_unit.properties.max_gems >
unit->s_unit.gems)
                        ? (unit->s_unit.properties.max_gems - unit->s_unit.gems) : 0;
                if (room >= obj->s_deposit_gems_pile.gems && unit &&
unit->s_unit.properties.gems_per_cooldown_step > 0)
                {
                        res.weight = 2;
                        res.action = CORE_TRAVEL_ATTACK;
                }
                else
                {
                        res.weight = 10000;
                        res.action = CORE_TRAVEL_BLOCK;
                }
                return res;
        }

        if (obj->type == OBJ_WALL || obj->type == OBJ_DEPOSIT)
        {
                if (unit && unit->s_unit.properties.damage_object > 0)
                {
                        res.weight = 2 + (obj->hp + unit->s_unit.properties.damage_object - 1) /
unit->s_unit.properties.damage_object;
                        res.action = CORE_TRAVEL_ATTACK;
                }
                else
                {
                        res.weight = 10000;
                        res.action = CORE_TRAVEL_BLOCK;
                }
                return res;
        }

        if (obj->type == OBJ_UNIT)
        {
                if (obj->s_unit.team_id == game.my_team_id)
                {
                        // BUG #6 FIX: Only block immediate next step (distance 1).
                        // Friendly units at distance >= 2 will vacate future corridor tiles!
                        if (unit && ft_distance(unit->pos, pos) <= 1)
                        {
```

```
                        res.weight = 10000;
                        res.action = CORE_TRAVEL_BLOCK;
                }
                else
                {
                        res.weight = 8;
                        res.action = CORE_TRAVEL_PASS;
                }
        }
        else if (unit && unit->s_unit.properties.damage_unit > 0)
        {
                res.weight = 2 + (obj->hp + unit->s_unit.properties.damage_unit - 1) /
unit->s_unit.properties.damage_unit;
                res.action = CORE_TRAVEL_ATTACK;
        }
        else
        {
                res.weight = 10000;
                res.action = CORE_TRAVEL_BLOCK;
        }
        return res;
    }

    if (obj->type == OBJ_CORE)
    {
        if (obj->s_core.team_id != game.my_team_id && unit &&
unit->s_unit.properties.damage_core > 0)
        {
                res.weight = 1;
                res.action = CORE_TRAVEL_ATTACK;
        }
        else
        {
                res.weight = 10000;
                res.action = CORE_TRAVEL_BLOCK;
        }
        return res;
    }

    return res;
}

void safe_travel(const t_obj *u, t_pos target)
```

```
{
	if (!u || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
		return;
	core_action_travel(u, target, safe_travelability);

	if (!has_unit_acted(u->id) && ft_distance(u->pos, target) >= 1)
	{
		const int dx[4] = {1, -1, 0, 0};
		const int dy[4] = {0, 0, 1, -1};
		unsigned int best_d = ft_distance(u->pos, target);
		t_pos best_step = u->pos;
		bool found_free = false;

		for (int d = 0; d < 4; d++)
		{
			int nx = (int)u->pos.x + dx[d];
			int ny = (int)u->pos.y + dy[d];
			if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
				continue;
			t_pos np = {(unsigned short)nx, (unsigned short)ny};
			t_obj *occ = core_get_obj_from_pos(np);
			if (occ && occ->type == OBJ_UNIT && occ->s_unit.team_id ==
game.my_team_id)
			{
				unsigned int dist = ft_distance(np, target);
				if (dist < best_d)
					nav_request_yield(np);
			}
			else if (!is_tile_occupied(np))
			{
				if (u->s_unit.properties.damage_unit == 0 &&
get_tile_enemy_danger(np) >= 500)
					continue;
				unsigned int dist = ft_distance(np, target);
				if (dist < best_d)
				{
					best_d = dist;
					best_step = np;
					found_free = true;
				}
			}
		}
		if (found_free && ft_distance(u->pos, target) > 1)
```

```
                        core_action_move(u, best_step);
        }
}

#define INF_DIST (UINT_MAX / 4)

void nav_compute_corridor_distances(t_pos start, unsigned int
dist[MAX_GRID_SIZE][MAX_GRID_SIZE],
                                                                bool is_unarmed, unsigned
long ignore_id)
{
        for (unsigned int y = 0; y < MAX_GRID_SIZE; y++)
                for (unsigned int x = 0; x < MAX_GRID_SIZE; x++)
                        dist[y][x] = INF_DIST;

        if (start.x >= game.grid_size || start.y >= game.grid_size)
                return;

        bool blocked[MAX_GRID_SIZE][MAX_GRID_SIZE] = {{false}};

        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                const t_obj *o = game.objects[i];
                if (o->pos.x >= game.grid_size || o->pos.y >= game.grid_size)
                        continue;
                if (o->type == OBJ_WALL || o->type == OBJ_DEPOSIT || o->type == OBJ_CORE ||
                        o->type == OBJ_GEM_PILE)
                {
                        blocked[o->pos.y][o->pos.x] = true;
                }
                else if (o->type == OBJ_UNIT && o->id != ignore_id)
                {
                        if (o->s_unit.team_id != game.my_team_id)
                                blocked[o->pos.y][o->pos.x] = true;
                }

                if (is_unarmed && o->type == OBJ_UNIT && o->s_unit.team_id !=
game.my_team_id &&
                        o->s_unit.properties.damage_unit > 0)
                {
                        for (int dy = -2; dy <= 2; dy++)
                                for (int dx = -2; dx <= 2; dx++)
                                {
```

```
                                        if (abs(dx) + abs(dy) <= 2)
                                        {
                                                int ex = (int)o->pos.x + dx;
                                                int ey = (int)o->pos.y + dy;
                                                if (ex >= 0 && ey >= 0 && ex < (int)game.grid_size &&
ey < (int)game.grid_size)
                                                        blocked[ey][ex] = true;
                                        }
                                }
                        }
                }

                t_pos queue[MAX_GRID_SIZE * MAX_GRID_SIZE];
                unsigned int head = 0, tail = 0;

                dist[start.y][start.x] = 0;
                queue[tail++] = start;

                const int dx[4] = {1, -1, 0, 0};
                const int dy[4] = {0, 0, 1, -1};

                while (head < tail)
                {
                        t_pos p = queue[head++];
                        for (int d = 0; d < 4; d++)
                        {
                                int nx = (int)p.x + dx[d];
                                int ny = (int)p.y + dy[d];
                                if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
                                        continue;
                                if (blocked[ny][nx] || dist[ny][nx] < INF_DIST)
                                        continue;
                                dist[ny][nx] = dist[p.y][p.x] + 1;
                                queue[tail++] = (t_pos){(unsigned short)nx, (unsigned short)ny};
                        }
                }
}

unsigned int nav_approach_distance(t_pos target, const unsigned int
dist[MAX_GRID_SIZE][MAX_GRID_SIZE])
{
        unsigned int best = INF_DIST;
        const int dx[4] = {1, -1, 0, 0};
```

```
	const int dy[4] = {0, 0, 1, -1};

	for (int d = 0; d < 4; d++)
	{
		int nx = (int)target.x + dx[d];
		int ny = (int)target.y + dy[d];
		if (nx >= 0 && ny >= 0 && nx < (int)game.grid_size && ny < (int)game.grid_size &&
			nx < MAX_GRID_SIZE && ny < MAX_GRID_SIZE)
		{
			if (dist[ny][nx] < best)
				best = dist[ny][nx];
		}
	}
	return best;
}

bool ft_carrier_can_reach_pile(const t_obj *pile, const t_obj *core)
{
	if (!core || !pile || pile->type != OBJ_GEM_PILE || pile->s_deposit_gems_pile.gems == 0)
		return false;
	if (ft_is_pos_near_armed_enemy(pile->pos, 3))
		return false;

	const t_obj *carrier = NULL;
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		const t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id == game.my_team_id &&
			o->s_unit.name && strcmp(o->s_unit.name, "Carrier") == 0)
		{
			carrier = o;
			break;
		}
	}
	if (!carrier)
		return false;

	static unsigned long cached_tick = ULONG_MAX;
	static unsigned long cached_id = 0;
	static t_pos cached_pos = {0, 0};
	static unsigned int out_dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
	static unsigned int home_dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
```

```
	if (cached_tick != game.elapsed_ticks || cached_id != carrier->id ||
		ft_distance(cached_pos, carrier->pos) > 0)
	{
		cached_tick = game.elapsed_ticks;
		cached_id = carrier->id;
		cached_pos = carrier->pos;
		nav_compute_corridor_distances(carrier->pos, out_dist, true, carrier->id);
		nav_compute_corridor_distances(core->pos, home_dist, true, carrier->id);
	}

	unsigned int app_out = nav_approach_distance(pile->pos, out_dist);
	unsigned int app_home = nav_approach_distance(pile->pos, home_dist);
	return (app_out < INF_DIST && app_home < INF_DIST);
}

unsigned long ft_runner_opportunity(const t_obj *core)
{
	if (!core || game.grid_size > MAX_GRID_SIZE)
		return 0;

	unsigned long guaranteed = ULONG_MAX;
	const int dx[4] = {1, -1, 0, 0};
	const int dy[4] = {0, 0, 1, -1};

	for (int d = 0; d < 4; d++)
	{
		int sx = (int)core->pos.x + dx[d];
		int sy = (int)core->pos.y + dy[d];
		if (sx < 0 || sy < 0 || sx >= (int)game.grid_size || sy >= (int)game.grid_size)
			continue;
		t_pos spawn_pos = {(unsigned short)sx, (unsigned short)sy};
		t_obj *occ = core_get_obj_from_pos(spawn_pos);
		if (occ && (occ->type == OBJ_WALL || occ->type == OBJ_DEPOSIT || occ->type
== OBJ_CORE))
			continue;

		unsigned int dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
		nav_compute_corridor_distances(spawn_pos, dist, true, 0);

		unsigned long exit_gems = 0;
		for (int i = 0; game.objects && game.objects[i]; i++)
		{
			t_obj *o = game.objects[i];
```

```
                        if (o->type != OBJ_GEM_PILE || o->s_deposit_gems_pile.gems == 0)
                                continue;
                        if (ft_is_pos_near_armed_enemy(o->pos, 3))
                                continue;
                        unsigned int app = nav_approach_distance(o->pos, dist);
                        if (app <= 24)
                                exit_gems += o->s_deposit_gems_pile.gems;
                }
                if (exit_gems < guaranteed)
                        guaranteed = exit_gems;
        }
        return (guaranteed == ULONG_MAX) ? 0 : guaranteed;
}

void nav_compute_finisher_lane(t_pos start, t_pos goal, bool
lane[MAX_GRID_SIZE][MAX_GRID_SIZE])
{
        memset(lane, 0, sizeof(bool) * MAX_GRID_SIZE * MAX_GRID_SIZE);
        if (!is_pos_in_bounds(start) || !is_pos_in_bounds(goal))
                return;

        unsigned int dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
        t_pos parent[MAX_GRID_SIZE][MAX_GRID_SIZE];
        bool visited[MAX_GRID_SIZE][MAX_GRID_SIZE];

        for (unsigned int y = 0; y < game.grid_size && y < MAX_GRID_SIZE; y++)
        {
                for (unsigned int x = 0; x < game.grid_size && x < MAX_GRID_SIZE; x++)
                {
                        dist[y][x] = UINT_MAX / 2;
                        visited[y][x] = false;
                        parent[y][x] = (t_pos){65535, 65535};
                }
        }

        dist[start.y][start.x] = 0;
        const int dx[4] = {1, -1, 0, 0};
        const int dy[4] = {0, 0, 1, -1};

        for (unsigned int iter = 0; iter < game.grid_size * game.grid_size; iter++)
        {
                unsigned int min_d = UINT_MAX / 2;
                int bx = -1, by = -1;
```

```
for (unsigned int y = 0; y < game.grid_size && y < MAX_GRID_SIZE; y++)
{
	for (unsigned int x = 0; x < game.grid_size && x < MAX_GRID_SIZE; x++)
	{
		if (!visited[y][x] && dist[y][x] < min_d)
		{
			min_d = dist[y][x];
			bx = (int)x;
			by = (int)y;
		}
	}
}

if (bx == -1 || min_d >= UINT_MAX / 4)
	break;

visited[by][bx] = true;
t_pos curr_pos = {(unsigned short)bx, (unsigned short)by};
if (ft_distance(curr_pos, goal) == 1)
{
	t_pos trace = curr_pos;
	while (true)
	{
		lane[trace.y][trace.x] = true;
		if (trace.x == start.x && trace.y == start.y)
			break;
		t_pos p = parent[trace.y][trace.x];
		if (p.x == 65535 && p.y == 65535)
			break;
		trace = p;
	}
	return;
}

for (int d = 0; d < 4; d++)
{
	int nx = bx + dx[d];
	int ny = by + dy[d];
	if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
		continue;
	if (visited[ny][nx])
		continue;
```

```
                        t_pos np = {(unsigned short)nx, (unsigned short)ny};
                        t_obj *obj = core_get_obj_from_pos(np);
                        unsigned int weight = 1;
                        if (obj)
                        {
                                if (obj->type == OBJ_CORE)
                                        continue;
                                if (obj->type == OBJ_WALL || obj->type == OBJ_DEPOSIT)
                                        weight = 2 + (obj->hp + 2) / 3;
                                else if (obj->type == OBJ_GEM_PILE &&
obj->s_deposit_gems_pile.gems > 16)
                                        continue;
                        }
                        // Prefer diagonal approach at distance 2 so cardinal guard posts stay
free
                        if (ft_distance(np, goal) == 2 && (np.x == goal.x || np.y == goal.y))
                                weight += 50;

                        if (dist[by][bx] + weight < dist[ny][nx])
                        {
                                dist[ny][nx] = dist[by][bx] + weight;
                                parent[ny][nx] = curr_pos;
                        }
                }
        }
}

void nav_yield_for_workers(t_obj *core, t_obj *enemy_core, t_obj **units)
{
        if (!core || !units)
                return;

        bool has_workers = false;
        for (int i = 0; units && units[i]; i++)
        {
                if (units[i]->s_unit.name && (strcmp(units[i]->s_unit.name, "Carrier") == 0 ||
                        strcmp(units[i]->s_unit.name, "Miner") == 0))
                {
                        has_workers = true;
                        break;
                }
        }
```

```
	const int dx[4] = {1, -1, 0, 0};
	const int dy[4] = {0, 0, 1, -1};

	for (int i = 0; units && units[i]; i++)
	{
		t_obj *u = units[i];
		if (!u->s_unit.name || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
			continue;
		if (strcmp(u->s_unit.name, "Hunter") != 0 && strcmp(u->s_unit.name, "Striker") !=
0 &&
			strcmp(u->s_unit.name, "Defender") != 0 && strcmp(u->s_unit.name,
"Scout") != 0)
			continue;

		bool requested = (u->pos.x < MAX_GRID_SIZE && u->pos.y < MAX_GRID_SIZE &&
			g_yield_request[u->pos.x][u->pos.y]);
		bool blocking_spawn = (has_workers && ft_distance(u->pos, core->pos) == 1);

		if (!requested && !blocking_spawn)
			continue;

		if (ft_get_best_adjacent_enemy(u) != NULL)
			continue;

		unsigned int best_score = UINT_MAX;
		t_pos destination = u->pos;
		bool found = false;

		for (int d = 0; d < 4; d++)
		{
			int nx = (int)u->pos.x + dx[d];
			int ny = (int)u->pos.y + dy[d];
			if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
				continue;
			t_pos p = {(unsigned short)nx, (unsigned short)ny};
			if (is_tile_occupied(p))
				continue;

			unsigned int score = (enemy_core ? ft_distance(p, enemy_core->pos) : 0);
			if (ft_distance(p, core->pos) == 1)
				score += 500;
			if (p.x < MAX_GRID_SIZE && p.y < MAX_GRID_SIZE &&
```

```
g_yield_request[p.x][p.y])
                                score += 1000;

                        if (score < best_score)
                        {
                                best_score = score;
                                destination = p;
                                found = true;
                        }
                }

                if (found)
                {
                        core_action_move(u, destination);
                        core_debug_addObjectInfo(u, "Yielded corridor to (%u,%u)",
destination.x, destination.y);
                }
        }
}

t_obj *nav_find_highway_obstacle(t_pos start, t_pos goal)
{
        if (!is_pos_in_bounds(start) || !is_pos_in_bounds(goal))
                return NULL;

        unsigned int dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
        t_pos parent[MAX_GRID_SIZE][MAX_GRID_SIZE];
        bool visited[MAX_GRID_SIZE][MAX_GRID_SIZE];

        for (unsigned int y = 0; y < game.grid_size && y < MAX_GRID_SIZE; y++)
        {
                for (unsigned int x = 0; x < game.grid_size && x < MAX_GRID_SIZE; x++)
                {
                        dist[y][x] = UINT_MAX / 2;
                        visited[y][x] = false;
                        parent[y][x] = (t_pos){65535, 65535};
                }
        }

        dist[start.y][start.x] = 0;

        for (unsigned int iter = 0; iter < game.grid_size * game.grid_size; iter++)
        {
```

```
unsigned int min_d = UINT_MAX / 2;
int bx = -1, by = -1;

for (unsigned int y = 0; y < game.grid_size && y < MAX_GRID_SIZE; y++)
{
	for (unsigned int x = 0; x < game.grid_size && x < MAX_GRID_SIZE; x++)
	{
		if (!visited[y][x] && dist[y][x] < min_d)
		{
			min_d = dist[y][x];
			bx = (int)x;
			by = (int)y;
		}
	}
}

if (bx == -1 || min_d >= UINT_MAX / 4)
	break;

visited[by][bx] = true;
if (bx == (int)goal.x && by == (int)goal.y)
	break;

const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};

for (int d = 0; d < 4; d++)
{
	int nx = bx + dx[d];
	int ny = by + dy[d];
	if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
		continue;
	if (visited[ny][nx])
		continue;

	t_pos np = {(unsigned short)nx, (unsigned short)ny};
	t_obj *obj = core_get_obj_from_pos(np);

	unsigned int weight = 1;
	if (obj)
	{
		if (obj->type == OBJ_WALL || obj->type == OBJ_DEPOSIT)
			weight = 10 + (obj->hp * 2);
```

```
                        else if (obj->type == OBJ_CORE)
                        {
                                if (nx != (int)goal.x || ny != (int)goal.y)
                                        continue;
                        }
                        else if (obj->type == OBJ_UNIT)
                        {
                                if (obj->s_unit.team_id == game.my_team_id)
                                        weight = 2;
                                else
                                        weight = 5;
                        }
                }

                if (dist[by][bx] + weight < dist[ny][nx])
                {
                        dist[ny][nx] = dist[by][bx] + weight;
                        parent[ny][nx] = (t_pos){(unsigned short)bx, (unsigned short)by};
                }
        }
}

if (dist[goal.y][goal.x] >= UINT_MAX / 4)
        return NULL;

t_pos path[MAX_GRID_SIZE * MAX_GRID_SIZE];
unsigned int path_len = 0;
t_pos curr = goal;

while (curr.x != start.x || curr.y != start.y)
{
        if (path_len < MAX_GRID_SIZE * MAX_GRID_SIZE)
                path[path_len++] = curr;
        t_pos p = parent[curr.y][curr.x];
        if (p.x == 65535 && p.y == 65535)
                break;
        curr = p;
}

for (int i = (int)path_len - 1; i >= 0; i--)
{
        t_pos p = path[i];
        t_obj *o = core_get_obj_from_pos(p);
```

```
                if (o && (o->type == OBJ_WALL || o->type == OBJ_DEPOSIT))
                        return o;
        }

        return NULL;
}

bool nav_is_path_to_core_open(t_pos start, t_pos goal)
{
        return (nav_find_highway_obstacle(start, goal) == NULL);
}
```

3. src/carrier.c (or carrier.c)

This file replaces the overly conservative is_pile_contested_by_enemy check, incorporates cargo-weighted return trip math (trip + (home + 1) * loaded + 2), and returns immediately when the Carrier's haul unlocks the next unit or when no safe reachable piles remain[2].

C

```
#include "bot.h"

static bool is_pile_contested_by_enemy(t_pos pile_pos, unsigned int my_trip)
{
        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                const t_obj *enemy = game.objects[i];
                if (enemy->type != OBJ_UNIT || enemy->s_unit.team_id == game.my_team_id ||
enemy->hp == 0)
                        continue;
                unsigned int edist = ft_distance(enemy->pos, pile_pos);
                // Never walk into an armed enemy guarding the pile within 3 tiles
                if (enemy->s_unit.properties.damage_unit > 0 && edist <= 3)
                        return true;
                // For unarmed enemy collectors, compare actual ticks to arrive (including
enemy base cooldown)
                unsigned int enemy_cd = (enemy->s_unit.properties.base_action_cooldown > 0)
                        ? (unsigned int)enemy->s_unit.properties.base_action_cooldown : 1;
                unsigned int enemy_wait = (enemy->s_unit.action_cooldown > 0)
                        ? (unsigned int)enemy->s_unit.action_cooldown : 0;
                unsigned int enemy_ticks = (edist > 0 ? (edist - 1) * enemy_cd : 0) + enemy_wait;
                if (edist <= 2 && enemy_ticks < my_trip)
```

```
                return true;
        }
        return false;
}

static bool carrier_evade_threat(t_obj *u, t_obj *core)
{
        if (!u || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
                return false;

        const t_obj *closest_threat = NULL;
        unsigned int min_threat_dist = UINT_MAX;

        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                const t_obj *o = game.objects[i];
                if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
                        o->s_unit.properties.damage_unit > 0 && o->hp > 0)
                {
                        unsigned int d = ft_distance(u->pos, o->pos);
                        if (d < min_threat_dist)
                        {
                                min_threat_dist = d;
                                closest_threat = o;
                        }
                }
        }

        if (!closest_threat || min_threat_dist > 2)
                return false;

        if (core && u->s_unit.gems > 0)
        {
                if (ft_is_adjacent(u->pos, core->pos))
                {
                        core_action_transferGems(u, core->pos, u->s_unit.gems);
                        return true;
                }
                if (ft_distance(u->pos, core->pos) <= 2)
                {
                        safe_travel(u, core->pos);
                        return true;
                }
```

```
}

const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};

t_pos best_step = u->pos;
long best_score = -999999;
bool found = false;

for (int d = 0; d < 4; d++)
{
	int nx = (int)u->pos.x + dx[d];
	int ny = (int)u->pos.y + dy[d];
	if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
		continue;
	t_pos p = {(unsigned short)nx, (unsigned short)ny};
	if (is_tile_occupied(p))
		continue;

	unsigned int step_min_threat = UINT_MAX;
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		const t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
			o->s_unit.properties.damage_unit > 0 && o->hp > 0)
		{
			unsigned int ed = ft_distance(p, o->pos);
			if (ed < step_min_threat)
				step_min_threat = ed;
		}
	}

	if (step_min_threat <= 1 && min_threat_dist > 1)
		continue;

	long score = (long)(step_min_threat * 1000);

	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		const t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
			o->s_unit.properties.damage_unit > 0 && o->hp > 0)
		{
```

```
                        if (ft_distance(p, o->pos) <= 1 && o->s_unit.action_cooldown <= 0)
                            score -= 50000;
                    }
                }

                for (int i = 0; game.objects && game.objects[i]; i++)
                {
                    const t_obj *o = game.objects[i];
                    if (o->type == OBJ_UNIT && o->s_unit.team_id == game.my_team_id &&
                        o->s_unit.properties.damage_unit > 0 && o->hp > 0)
                    {
                        unsigned int ad = ft_distance(p, o->pos);
                        if (ad <= 6)
                            score += (long)((7 - ad) * 40);
                    }
                }

                if (core)
                {
                    unsigned int cd = ft_distance(p, core->pos);
                    score += (long)((30 - cd) * 15);
                    if (cd == 1)
                        score -= 80;
                }

                if (score > best_score)
                {
                    best_score = score;
                    best_step = p;
                    found = true;
                }
            }

            if (found && (best_step.x != u->pos.x || best_step.y != u->pos.y) && best_score > -40000)
            {
                core_action_move(u, best_step);
                return true;
            }

            return false;
}

void handle_carrier(t_obj *core, t_obj **units)
```

```
{
	if (!core)
		return;

	unsigned long claimed_piles[64];
	unsigned int n_claimed_piles = 0;

	for (int i = 0; units && units[i]; i++)
	{
		t_obj *u = units[i];
		if (!u->s_unit.name || strcmp(u->s_unit.name, "Carrier") != 0)
			continue;
		if (u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
			continue;

		if (carrier_evade_threat(u, core))
			continue;

		unsigned long carried = u->s_unit.gems;
		unsigned long capacity = u->s_unit.properties.max_gems;
		unsigned int n_miners = core_get_units_byName_count("Miner");

		// Evaluate best reachable safe gem pile FIRST using cargo-weighted trip cost
		t_obj *best_pile = NULL;
		long best_pile_score = LONG_MIN;
		unsigned int best_pile_trip = UINT_MAX;

		unsigned int dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
		unsigned int home_dist[MAX_GRID_SIZE][MAX_GRID_SIZE];
		nav_compute_corridor_distances(u->pos, dist, true, u->id);
		nav_compute_corridor_distances(core->pos, home_dist, true, u->id);

		if (carried < capacity)
		{
			for (int k = 0; game.objects && game.objects[k]; k++)
			{
				t_obj *o = game.objects[k];
				if (o->type != OBJ_GEM_PILE || o->s_deposit_gems_pile.gems ==
0)
					continue;
				if (ft_is_pos_near_armed_enemy(o->pos, 3))
					continue;
				if (o->s_deposit_gems_pile.gems <=
```

```
get_pending_pile_gems(o->id))
                                        continue;

                                bool already_claimed = false;
                                for (unsigned int c = 0; c < n_claimed_piles; c++)
                                {
                                        if (claimed_piles[c] == o->id)
                                        {
                                                already_claimed = true;
                                                break;
                                        }
                                }
                                if (already_claimed)
                                        continue;

                                unsigned int trip = nav_approach_distance(o->pos, dist);
                                unsigned int home = nav_approach_distance(o->pos, home_dist);
                                if (trip >= (UINT_MAX / 4) || home >= (UINT_MAX / 4))
                                        continue;
                                if (is_pile_contested_by_enemy(o->pos, trip))
                                        continue;

                                unsigned long space = capacity - carried;
                                unsigned long gain = (o->s_deposit_gems_pile.gems < space)
                                        ? o->s_deposit_gems_pile.gems : space;
                                unsigned long step =
u->s_unit.properties.gems_per_cooldown_step;
                                unsigned long base_cd =
(u->s_unit.properties.base_action_cooldown > 0)
                                        ? (unsigned
long)u->s_unit.properties.base_action_cooldown : 0;
                                unsigned long loaded = base_cd + (carried + gain) / (step ? step :
1);
                                if (loaded == 0)
                                        loaded = 1;

                                unsigned long total_cost = (unsigned long)trip + ((unsigned
long)home + 1) * loaded + 2;
                                long score = (long)((gain * 1000) / (total_cost ? total_cost : 1));
                                if (score > best_pile_score)
                                {
                                        best_pile_score = score;
                                        best_pile = o;
```

```
                                best_pile_trip = trip;
                        }
                }
        }

        // BUG #4 FIX: Deliver immediately if full, if no safe pile exists, if haul unlocks next
unit,
        // or if already carrying >= 50 gems and the next pile is far away (> 10 tiles)
        unsigned long needed = (n_miners == 0) ? COST_DIGGER : COST_SWORD;
        unsigned long core_gems = core->s_core.gems + get_pending_core_gems();
        bool need_delivery = false;

        if (carried > 0 && (carried >= 96 || carried >= capacity || !best_pile ||
                (carried >= 36 && core_gems < needed && core_gems + carried >=
needed) ||
                (carried >= 50 && best_pile_trip > 10)))
        {
                need_delivery = true;
        }

        if (need_delivery)
        {
                if (ft_is_adjacent(u->pos, core->pos))
                {
                        core_action_transferGems(u, core->pos, carried);
                        continue;
                }
                safe_travel(u, core->pos);
                continue;
        }

        if (best_pile)
        {
                if (n_claimed_piles < 64)
                        claimed_piles[n_claimed_piles++] = best_pile->id;

                if (ft_is_adjacent(u->pos, best_pile->pos))
                {
                        core_action_attack(u, best_pile);
                        if (!has_unit_acted(u->id) && carried > 0)
                                safe_travel(u, core->pos);
                }
                else
```

```
                {
                        safe_travel(u, best_pile->pos);
                }
                continue;
        }

        if (carried > 0)
        {
                if (ft_is_adjacent(u->pos, core->pos))
                {
                        core_action_transferGems(u, core->pos, carried);
                        continue;
                }
                safe_travel(u, core->pos);
                continue;
        }

        // Park at distance 2 from Core while waiting for Digger to crack the next Deposit
        if (ft_distance(u->pos, core->pos) == 1)
                step_away_from_core(u, core);
    }
}
```

4. src/miners.c (or miners.c)

This file eliminates the primary economy bottleneck in your bot: when a Carrier is active, your fast Digger (max_gems <= 20) **never scoops gems from a broken Deposit's gem pile if the Carrier can reach it**[9]. Instead, the Digger leaves the 150-gem pile on the ground for the 1-tick Carrier and immediately moves to break the next Deposit[9].

C

```
#include "bot.h"

static unsigned long g_sticky_highway_id = 0;

static bool miner_evade_threat(t_obj *u, t_obj *core, t_obj **units)
{
    (void)core;
    if (!u || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
        return false;
```

```
	const t_obj *closest_threat = NULL;
	unsigned int min_threat_dist = UINT_MAX;

	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		const t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
			o->s_unit.properties.damage_unit > 0 && o->hp > 0)
		{
			unsigned int d = ft_distance(u->pos, o->pos);
			if (d < min_threat_dist)
			{
				min_threat_dist = d;
				closest_threat = o;
			}
		}
	}

	if (!closest_threat || min_threat_dist > 2)
		return false;

	if (ft_count_allies_near_pos(closest_threat->pos, 1) > 0)
		return false;

	if (u->s_unit.gems > 0)
	{
		for (int j = 0; units && units[j]; j++)
		{
			t_obj *c = units[j];
			if (c->s_unit.name && strcmp(c->s_unit.name, "Carrier") == 0 &&
ft_is_adjacent(u->pos, c->pos))
			{
				unsigned long max_g = c->s_unit.properties.max_gems;
				unsigned long curr_g = c->s_unit.gems;
				if (curr_g < max_g)
				{
					unsigned long space = max_g - curr_g;
					unsigned long amt = (u->s_unit.gems < space) ?
u->s_unit.gems : space;
					core_action_transferGems(u, c->pos, amt);
					return true;
				}
```

```
                }
        }
}

const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};
t_pos best_step = u->pos;
long best_score = -999999;
bool found = false;

for (int d = 0; d < 4; d++)
{
        int nx = (int)u->pos.x + dx[d];
        int ny = (int)u->pos.y + dy[d];
        if (nx < 0 || ny < 0 || nx >= (int)game.grid_size || ny >= (int)game.grid_size)
                continue;
        t_pos p = {(unsigned short)nx, (unsigned short)ny};
        if (is_tile_occupied(p))
                continue;

        unsigned int step_min_threat = UINT_MAX;
        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                const t_obj *o = game.objects[i];
                if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
                        o->s_unit.properties.damage_unit > 0 && o->hp > 0)
                {
                        unsigned int ed = ft_distance(p, o->pos);
                        if (ed < step_min_threat)
                                step_min_threat = ed;
                }
        }

        long score = (long)(step_min_threat * 1000);

        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                const t_obj *o = game.objects[i];
                if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
                        o->s_unit.properties.damage_unit > 0 && o->hp > 0)
                {
                        if (ft_distance(p, o->pos) <= 1 && o->s_unit.action_cooldown <= 0)
                                score -= 50000;
```

```
                        }
                }

                if (score > best_score)
                {
                        best_score = score;
                        best_step = p;
                        found = true;
                }
        }

        if (found && (best_step.x != u->pos.x || best_step.y != u->pos.y) && best_score > -40000)
        {
                core_action_move(u, best_step);
                return true;
        }

        return false;
}

void handle_miners(t_obj *core, t_obj **units)
{
        if (!core)
                return;

        unsigned int n_carriers = core_get_units_byName_count("Carrier");
        t_obj *opp_core = ft_get_core_opponent();

        for (int i = 0; units && units[i]; i++)
        {
                t_obj *u = units[i];
                if (!u->s_unit.name || strcmp(u->s_unit.name, "Miner") != 0)
                        continue;
                if (u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
                        continue;

                if (miner_evade_threat(u, core, units))
                        continue;

                unsigned long carried = u->s_unit.gems;

                // 1. If holding any gems and adjacent to Carrier or Core, transfer them
immediately
```

```
		if (carried > 0)
		{
			if (ft_is_adjacent(u->pos, core->pos))
			{
				core_action_transferGems(u, core->pos, carried);
				continue;
			}
			for (int j = 0; units && units[j]; j++)
			{
				t_obj *c = units[j];
				if (c->s_unit.name && strcmp(c->s_unit.name, "Carrier") == 0 &&
					ft_is_adjacent(u->pos, c->pos) &&
					c->s_unit.gems < c->s_unit.properties.max_gems)
				{
					unsigned long space = c->s_unit.properties.max_gems -
c->s_unit.gems;
					unsigned long amt = (carried < space) ? carried : space;
					core_action_transferGems(u, c->pos, amt);
					break;
				}
			}
			if (has_unit_acted(u->id))
				continue;

			// Return to Core if holding gems (from a walled-off pile or solo miner
mode)
			if (n_carriers == 0 ? (carried >= 48 || carried >=
u->s_unit.properties.max_gems) : (carried > 0))
			{
				safe_travel(u, core->pos);
				continue;
			}
		}

		// 2. Attack adjacent Deposit FIRST
		t_obj *dep = ft_get_deposit_nearest(u->pos);
		if (dep && ft_is_adjacent(u->pos, dep->pos))
		{
			core_action_attack(u, dep);
			continue;
		}

		// 3. BUG #1 FIX: Only attack an adjacent Gem Pile if NO Carrier can collect it!
```

```
		t_obj *pile = ft_get_gems_nearest(u->pos);
		if (pile && ft_is_adjacent(u->pos, pile->pos) &&
			u->s_unit.gems < u->s_unit.properties.max_gems &&
			pile->s_deposit_gems_pile.gems > get_pending_pile_gems(pile->id) &&
			(n_carriers == 0 || !ft_carrier_can_reach_pile(pile, core)))
		{
			core_action_attack(u, pile);
			continue;
		}

		// 4. Find assigned deconflicted resource
		t_obj *target = ft_get_resource_for_miner(u->pos, core->pos, u->id);
		if (target)
		{
			if (ft_is_adjacent(u->pos, target->pos))
				core_action_attack(u, target);
			else
				safe_travel(u, target->pos);
			continue;
		}

		// 5. Highway Sapper: clear obstacles blocking the path between Cores
		t_obj *highway_obs = NULL;
		if (g_sticky_highway_id != 0)
		{
			t_obj *st = core_get_obj_from_id(g_sticky_highway_id);
			if (st && st->hp > 0 && (st->type == OBJ_WALL || st->type ==
OBJ_DEPOSIT))
				highway_obs = st;
			else
				g_sticky_highway_id = 0;
		}

		if (!highway_obs && opp_core)
		{
			highway_obs = nav_find_highway_obstacle(core->pos, opp_core->pos);
			if (highway_obs)
				g_sticky_highway_id = highway_obs->id;
		}

		if (highway_obs)
		{
			if (ft_is_adjacent(u->pos, highway_obs->pos))
```

```
                        core_action_attack(u, highway_obs);
                    else
                        safe_travel(u, highway_obs->pos);
                    continue;
                }

                if (carried > 0)
                {
                    if (ft_is_adjacent(u->pos, core->pos))
                        core_action_transferGems(u, core->pos, carried);
                    else
                        safe_travel(u, core->pos);
                    continue;
                }

                if (ft_is_adjacent(u->pos, core->pos))
                    step_away_from_core(u, core);
            }
        }
```

5. src/combat.c (or combat.c)

This file implements two key combat upgrades from Draft (sword.c and flank_breaker.c). First, **Bounded Open-Gap First-Strike Ambush** ensures Hunter units only hold at Manhattan distance 2 when ft_has_open_melee_gap(u, e) confirms an unobstructed shared cardinal tile exists between them and the enemy, while get_unit_wait_ticks bounds the hold to under 6 ticks so readiness never wraps[8]. Second, **Endgame Finisher Lane and Flanking Guard Posts** ensure that once enemy units are cleared, nav_compute_finisher_lane marks the shortest approach corridor to the enemy Core[1]. Hunter units automatically step off that corridor and claim flanking guard posts at distance 2–4 around the enemy Core, giving your Striker an open lane to deal 6 Core damage per hit while your Hunter pack spawn-camps any newly created enemy unit[8].

C

```
#include "bot.h"

static bool g_seen_enemy_unit = false;

typedef struct s_wait_entry
{
```

```
	unsigned long unit_id;
	unsigned long since_tick;
	bool active;
} t_wait_entry;

static t_wait_entry g_waits[64];

static unsigned long get_unit_wait_ticks(unsigned long unit_id, bool should_wait)
{
	int slot = -1;
	for (int i = 0; i < 64; i++)
	{
		if (g_waits[i].active && g_waits[i].unit_id == unit_id)
		{
			slot = i;
			break;
		}
		if (!g_waits[i].active && slot == -1)
			slot = i;
	}
	if (!should_wait)
	{
		if (slot != -1 && g_waits[slot].unit_id == unit_id)
			g_waits[slot].active = false;
		return 0;
	}
	if (slot != -1)
	{
		if (!g_waits[slot].active || g_waits[slot].unit_id != unit_id)
		{
			g_waits[slot].unit_id = unit_id;
			g_waits[slot].since_tick = game.elapsed_ticks;
			g_waits[slot].active = true;
		}
		return (game.elapsed_ticks - g_waits[slot].since_tick);
	}
	return 0;
}

bool ft_enemy_units_cleared(void)
{
	bool enemy_alive = false;
	for (int i = 0; game.objects && game.objects[i]; i++)
```

```
	{
		const t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id && o->hp >
0)
		{
			g_seen_enemy_unit = true;
			enemy_alive = true;
		}
	}
	return (g_seen_enemy_unit && !enemy_alive) || (game.elapsed_ticks > 400 &&
!enemy_alive);
}

static void handle_defender(t_obj *u, t_obj *core, t_obj *opp_core, t_game_phase phase)
{
	if (!u || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
		return;
	if (!core)
		return;

	t_obj *adj_opp = ft_get_best_adjacent_enemy(u);
	if (adj_opp)
	{
		core_action_attack(u, adj_opp);
		return;
	}

	t_obj *threat = ft_get_threat_nearest(core->pos, DEFENDER_INTERCEPT_RADIUS);
	if (threat)
	{
		if (ft_is_adjacent(u->pos, threat->pos))
			core_action_attack(u, threat);
		else if (!ft_should_hold_for_first_strike(u, threat))
			safe_travel(u, threat->pos);
		return;
	}

	if (phase == PHASE_3_ALL_OUT_ASSAULT || !opp_core || opp_core->hp == 0)
	{
		t_obj *opp = ft_get_units_opponent_nearest(u->pos);
		if (opp)
		{
			if (ft_is_adjacent(u->pos, opp->pos))
```

```
                        core_action_attack(u, opp);
                    else if (!ft_should_hold_for_first_strike(u, opp))
                        safe_travel(u, opp->pos);
                    return;
                }
        }

        t_pos choke1 = core->pos;
        t_pos choke2 = core->pos;
        if (core->pos.x == 0 && core->pos.y == 0)
        {
                choke1.x = 2; choke1.y = 0;
                choke2.x = 0; choke2.y = 2;
        }
        else
        {
                choke1.x = (core->pos.x >= 2) ? core->pos.x - 2 : 0;
                choke1.y = core->pos.y;
                choke2.x = core->pos.x;
                choke2.y = (core->pos.y >= 2) ? core->pos.y - 2 : 0;
        }

        t_pos choke = (u->id % 2 == 0) ? choke1 : choke2;
        t_obj *occ = core_get_obj_from_pos(choke);
        if (occ && occ->id != u->id)
                choke = (u->id % 2 == 0) ? choke2 : choke1;

        if (ft_distance(u->pos, choke) > 0)
                safe_travel(u, choke);
}

// Assigns Hunters to guard posts at distance 2-4 around enemy Core OUTSIDE the Finisher's
lane!
static void position_hunter_around_enemy_core(t_obj *u, t_obj *opp_core,
                                                                        const bool
lane[MAX_GRID_SIZE][MAX_GRID_SIZE],
                                                                        bool
claimed_posts[MAX_GRID_SIZE][MAX_GRID_SIZE])
{
        if (!u || !opp_core)
                return;

        unsigned int d_opp = ft_distance(u->pos, opp_core->pos);
```

```
if (u->s_unit.properties.damage_core > 0 && d_opp == 1)
{
	core_action_attack(u, opp_core);
	return;
}

t_pos best_post = u->pos;
unsigned int best_score = UINT_MAX;
bool found = false;

for (unsigned short y = 0; y < game.grid_size && y < MAX_GRID_SIZE; y++)
{
	for (unsigned short x = 0; x < game.grid_size && x < MAX_GRID_SIZE; x++)
	{
		t_pos p = {x, y};
		unsigned int d = ft_distance(p, opp_core->pos);
		if (d < 2 || d > 4 || lane[y][x] || claimed_posts[y][x])
			continue;

		t_obj *occ = core_get_obj_from_pos(p);
		if (occ && occ->id != u->id)
			continue;
		if (is_tile_occupied(p) && (p.x != u->pos.x || p.y != u->pos.y))
			continue;

		unsigned int score = (d - 2) * 300 + ft_distance(u->pos, p) * 10;
		// Prefer cardinal distance-2 posts that cover enemy spawn exits
		if (d == 2 && (p.x == opp_core->pos.x || p.y == opp_core->pos.y))
			score = (score > 100) ? (score - 100) : 0;

		if (score < best_score)
		{
			best_score = score;
			best_post = p;
			found = true;
		}
	}
}

if (found)
{
	claimed_posts[best_post.y][best_post.x] = true;
	if (best_post.x != u->pos.x || best_post.y != u->pos.y)
```

```
                safe_travel(u, best_post);
        }
        else if (d_opp == 1 || (u->pos.x < MAX_GRID_SIZE && u->pos.y < MAX_GRID_SIZE &&
lane[u->pos.y][u->pos.x]))
        {
                step_away_from_core(u, opp_core);
        }
}

static void handle_hunter_or_scout(t_obj *u, t_obj *core, t_obj *opp_core,
                                                                t_game_phase phase,
unsigned long shared_focus_id,
                                                                const bool
lane[MAX_GRID_SIZE][MAX_GRID_SIZE],
                                                                bool
claimed_posts[MAX_GRID_SIZE][MAX_GRID_SIZE])
{
        (void)phase;
        if (!u || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
                return;

        // 1. Attack best adjacent enemy immediately (executes in Stage 3 before movement)
        t_obj *adj_opp = ft_get_best_adjacent_enemy(u);
        if (adj_opp)
        {
                get_unit_wait_ticks(u->id, false);
                core_action_attack(u, adj_opp);
                return;
        }

        // 2. Select highest-priority enemy unit (with Pack Focus-Fire)
        t_obj **opps = ft_get_units_opponent();
        t_obj *best_target = NULL;
        long best_priority = LONG_MIN;

        for (int i = 0; opps && opps[i]; i++)
        {
                t_obj *e = opps[i];
                if (get_pending_damage(e->id) >= e->hp)
                        continue;

                unsigned int dist_me = ft_distance(u->pos, e->pos);
                unsigned int dist_core = core ? ft_distance(core->pos, e->pos) : 20;
```

```
                bool is_unarmed = (e->s_unit.properties.damage_unit == 0 &&
e->s_unit.properties.damage_core == 0);

                long priority = 1000 - (long)(dist_me * 50) - (long)(e->hp * 5);

                if (e->id == shared_focus_id && dist_me <= 8)
                        priority += 700;
                if (ft_is_enemy_engaged_by_ally(e, u->id))
                        priority += 900;
                if (u->s_unit.properties.damage_unit >= e->hp)
                        priority += 1000;

                if (is_unarmed)
                        priority += 250 + (long)(e->s_unit.gems * 2);
                else if (dist_me <= 5)
                        priority += 500;

                if (core && dist_core <= 6 && dist_me <= 10 && !is_unarmed)
                        priority += 5000;

                if (opp_core && core_get_units_byName_count("Striker") > 0 &&
                        ft_distance(e->pos, opp_core->pos) <= 2)
                        priority += 2500;

                if (priority > best_priority)
                {
                        best_priority = priority;
                        best_target = e;
                }
        }
        free(opps);

        if (best_target)
        {
                if (ft_is_adjacent(u->pos, best_target->pos))
                {
                        get_unit_wait_ticks(u->id, false);
                        core_action_attack(u, best_target);
                        return;
                }

                // BUG #7 FIX: Bounded Distance-2 First-Strike Ambush with Open Melee Gap
verification
```

```
                bool raw_hold = false;
                unsigned long step = u->s_unit.properties.gems_per_cooldown_step;
                long move_cd = (long)u->s_unit.properties.base_action_cooldown +
                        (long)(u->s_unit.gems / (step ? step : 1));
                if (move_cd < 1)
                        move_cd = 1;

                for (int j = 0; game.objects && game.objects[j]; j++)
                {
                        const t_obj *e = game.objects[j];
                        if (e->type == OBJ_UNIT && e->s_unit.team_id != game.my_team_id &&
                                e->s_unit.properties.damage_unit > 0 && e->hp > 0 &&
                                ft_distance(u->pos, e->pos) == 2 &&
                                e->s_unit.action_cooldown < move_cd &&
                                ft_has_open_melee_gap(u, e) &&
                                !ft_is_enemy_engaged_by_ally(e, u->id))
                        {
                                raw_hold = true;
                                break;
                        }
                }

                unsigned long waited = get_unit_wait_ticks(u->id, raw_hold);
                if (raw_hold && waited < 6)
                {
                        if (core && ft_is_adjacent(u->pos, core->pos))
                                step_away_from_core(u, core);
                        return;
                }

                safe_travel(u, best_target->pos);
                return;
        }

        // 3. BUG #2 FIX: When enemy units are cleared, guard enemy Core at distance 2-4
        // OUTSIDE the Finisher's corridor so Striker can march straight to distance 1!
        get_unit_wait_ticks(u->id, false);
        if (opp_core)
                position_hunter_around_enemy_core(u, opp_core, lane, claimed_posts);
}

static void handle_striker(t_obj *u, t_obj *core, t_obj *opp_core)
{
```

```
	(void)core;
	if (!u || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
		return;
	if (!opp_core || opp_core->hp == 0)
		return;

	if (ft_is_adjacent(u->pos, opp_core->pos))
	{
		core_action_attack(u, opp_core);
		return;
	}

	safe_travel(u, opp_core->pos);
}

void handle_combat(t_obj *core, t_obj *opp_core, t_obj **units)
{
	t_game_phase phase = ft_get_game_phase(core, opp_core);

	unsigned long shared_focus_id = 0;
	if (core)
	{
		t_obj *threat = ft_get_units_opponent_nearest(core->pos);
		if (threat)
			shared_focus_id = threat->id;
	}

	// Compute Finisher lane from active Striker (or our Core) to enemy Core
	bool finisher_lane[MAX_GRID_SIZE][MAX_GRID_SIZE];
	bool claimed_posts[MAX_GRID_SIZE][MAX_GRID_SIZE] = {{false}};
	memset(finisher_lane, 0, sizeof(finisher_lane));

	if (core && opp_core)
	{
		t_pos lane_start = core->pos;
		for (int i = 0; units && units[i]; i++)
		{
			if (units[i]->s_unit.name && strcmp(units[i]->s_unit.name, "Striker") == 0)
			{
				lane_start = units[i]->pos;
				break;
			}
		}
```

```
            nav_compute_finisher_lane(lane_start, opp_core->pos, finisher_lane);
        }

        // Run Strikers FIRST when enemy units are cleared so they claim the approach tile
        if (ft_enemy_units_cleared())
        {
            for (int i = 0; units && units[i]; i++)
            {
                t_obj *u = units[i];
                if (u->s_unit.name && strcmp(u->s_unit.name, "Striker") == 0)
                    handle_striker(u, core, opp_core);
            }
        }

        for (int i = 0; units && units[i]; i++)
        {
            t_obj *u = units[i];
            if (!u->s_unit.name || u->s_unit.action_cooldown > 0 || has_unit_acted(u->id))
                continue;

            if (strcmp(u->s_unit.name, "Defender") == 0)
                handle_defender(u, core, opp_core, phase);
            else if (strcmp(u->s_unit.name, "Hunter") == 0 || strcmp(u->s_unit.name, "Scout")
== 0)
                handle_hunter_or_scout(u, core, opp_core, phase, shared_focus_id,
finisher_lane, claimed_posts);
            else if (strcmp(u->s_unit.name, "Striker") == 0)
                handle_striker(u, core, opp_core);
        }
}
```

6. src/spawner.c (or spawner.c)

This file removes the g_next_allowed_spawn_tick multi-tick lockout, accounts for same-tick gem transfers (core->s_core.gems + get_pending_core_gems()), and adds an emergency defense fallback so your Core does not sit idle with unspent gems if an enemy rusher reaches distance 4 before you have 307 gems[3].

C

```
#include "bot.h"
```

```
static unsigned long g_last_spawn_tick = ULONG_MAX;
static bool g_had_economy = false;
static bool g_attacked = false;

static void safe_create_unit(t_obj *core, const char *name, unsigned long cost,
        char *c1, char *c2, char *c3, char *c4, char *c5, char *c6)
{
        if (!core || core->s_core.spawn_cooldown > 0)
                return;
        if (game.elapsed_ticks == g_last_spawn_tick)
                return;
        if (core->s_core.gems + get_pending_core_gems() < cost)
                return;
        if (!has_free_spawn_tile(core->pos))
                return;

        g_last_spawn_tick = game.elapsed_ticks;
        core_action_createUnit(name, c1, c2, c3, c4, c5, c6, NULL);
}

static void spawn_brawler_or_hunter(t_obj *core, unsigned long gems, unsigned int n_hunters,
bool emergency)
{
        // Primary Meta Fighter: 22 HP, 3-tick cooldown, 3 DMG (307 gems)
        if (gems >= COST_BRAWLER_HEAVY)
        {
                safe_create_unit(core, "Hunter", COST_BRAWLER_HEAVY, "combat", "health",
"health", "health", "speed", "speed");
                return;
        }
        // Emergency panic fallback ONLY if enemy is right at our Core (dist <= 4) and we have 0
fighters
        if (emergency && n_hunters == 0 && ft_is_core_threatened(core, 4))
        {
                if (gems >= COST_SKIRMISHER)
                        safe_create_unit(core, "Hunter", COST_SKIRMISHER, "combat", "health",
"speed", "speed", NULL, NULL);
                else if (gems >= COST_HUNTER_TANK)
                        safe_create_unit(core, "Hunter", COST_HUNTER_TANK, "combat",
"combat", "health", NULL, NULL, NULL);
        }
}
```

```
void handle_spawner(t_obj *core)
{
	if (!core || core->s_core.spawn_cooldown > 0)
		return;
	if (!has_free_spawn_tile(core->pos))
		return;

	unsigned long gems = core->s_core.gems + get_pending_core_gems();
	unsigned int n_carriers = core_get_units_byName_count("Carrier");
	unsigned int n_miners = core_get_units_byName_count("Miner");
	unsigned int n_defenders = core_get_units_byName_count("Defender");
	unsigned int n_hunters = core_get_units_byName_count("Hunter");
	unsigned int n_strikers = core_get_units_byName_count("Striker");

	if (n_carriers > 0 || n_miners > 0)
		g_had_economy = true;
	if (n_hunters > 0)
		g_attacked = true;

	unsigned int safe_res = ft_count_safe_resources(core->pos);
	bool threatened = ft_is_core_threatened(core, 6);
	unsigned long loose = ft_runner_opportunity(core);
	bool enemy_cleared = ft_enemy_units_cleared();

	// 1. Opening Hyper-Runner when corridors connect to loose gems
	if (!g_had_economy && n_carriers == 0 && n_miners == 0 && loose > 0 &&
		game.elapsed_ticks <= 350 && !threatened)
	{
		if (gems >= COST_RUNNER)
			safe_create_unit(core, "Carrier", COST_RUNNER, "max_gems",
"gem_speed", "speed", "speed", NULL, NULL);
		return;
	}

	// 2. Emergency Core Defense: if threatened and we have no fighters yet
	if (threatened && (n_hunters + n_defenders == 0) && gems >= COST_BRAWLER_HEAVY)
	{
		spawn_brawler_or_hunter(core, gems, n_hunters, true);
		return;
	}
	if (ft_is_core_threatened(core, 4) && (n_hunters + n_defenders == 0) && gems >=
COST_HUNTER_TANK)
	{
```

```
                spawn_brawler_or_hunter(core, gems, n_hunters, true);
                return;
        }

        // 3. Ensure at least 1 Carrier is active whenever loose gems exist
        if (n_carriers < 1 && loose > 0)
        {
                if (gems >= COST_RUNNER)
                {
                        safe_create_unit(core, "Carrier", COST_RUNNER, "max_gems",
"gem_speed", "speed", "speed", NULL, NULL);
                        return;
                }
                if (game.elapsed_ticks > 350 && gems >= 152)
                {
                        safe_create_unit(core, "Carrier", 152, "speed", "speed", NULL, NULL, NULL,
NULL);
                        return;
                }
        }

        // 4. First Fast Digger (or solo Miner if no Carrier exists)
        if (n_miners < 1 && safe_res > 0)
        {
                if (n_carriers > 0 && gems >= COST_DIGGER)
                        safe_create_unit(core, "Miner", COST_DIGGER, "demolition", "speed",
"speed", NULL, NULL, NULL);
                else if (n_carriers == 0 && gems >= COST_MINER_SOLO)
                        safe_create_unit(core, "Miner", COST_MINER_SOLO, "demolition",
"max_gems", NULL, NULL, NULL, NULL);
                else if (game.elapsed_ticks > 350 && gems >= 77)
                        safe_create_unit(core, "Miner", 77, "demolition", NULL, NULL, NULL, NULL,
NULL);
                return;
        }

        // 5. Second Fast Digger once we have attacked (at least 1 Hunter built) and 2+ safe
deposits exist
        if (n_miners == 1 && n_carriers > 0 && g_attacked && safe_res >= 2 &&
                !ft_is_core_threatened(core, 10))
        {
                if (gems >= COST_DIGGER)
                        safe_create_unit(core, "Miner", COST_DIGGER, "demolition", "speed",
```

```
"speed", NULL, NULL, NULL);
                return;
        }

        // 6. Core Finisher once enemy units are cleared and we have Hunters guarding the
enemy Core
        if (n_hunters >= 1 && enemy_cleared)
        {
                if (n_strikers < 1)
                {
                        if (gems >= COST_FINISHER)
                                safe_create_unit(core, "Striker", COST_FINISHER, "siege", "siege",
"demolition", "health", NULL, NULL);
                        return;
                }
        }

        // 7. Continuous Heavy Brawler production
        spawn_brawler_or_hunter(core, gems, n_hunters, threatened);
}
```

7. src/getters.c (or getters.c)

Add ft_has_open_melee_gap to getters.c and update ft_get_resource_for_miner so Miners prioritize Deposits and only target Gem Piles that the Carrier cannot reach[8].

C

```
#include "bot.h"

unsigned int ft_distance(t_pos a, t_pos b)
{
        unsigned int dx = (a.x > b.x) ? (a.x - b.x) : (b.x - a.x);
        unsigned int dy = (a.y > b.y) ? (a.y - b.y) : (b.y - a.y);
        return (dx + dy);
}

bool ft_is_adjacent(t_pos a, t_pos b)
{
        return (ft_distance(a, b) <= 1);
}
```

```
static bool is_core(const t_obj *obj) { return (obj->type == OBJ_CORE); }
static bool is_core_own(const t_obj *obj) { return (is_core(obj) && obj->s_core.team_id ==
game.my_team_id); }
static bool is_core_opponent(const t_obj *obj) { return (is_core(obj) && obj->s_core.team_id !=
game.my_team_id); }
static bool is_deposit(const t_obj *obj) { return (obj->type == OBJ_DEPOSIT); }
static bool is_gems(const t_obj *obj) { return (obj->type == OBJ_GEM_PILE); }
static bool is_wall(const t_obj *obj) { return (obj->type == OBJ_WALL); }
static bool is_unit(const t_obj *obj) { return (obj->type == OBJ_UNIT); }
static bool is_unit_own(const t_obj *obj) { return (is_unit(obj) && obj->s_unit.team_id ==
game.my_team_id); }
static bool is_unit_opponent(const t_obj *obj) { return (is_unit(obj) && obj->s_unit.team_id !=
game.my_team_id); }

t_obj *ft_get_core_own(void)
{
	t_pos pos = {0, 0};
	return core_get_obj_filter_nearest(pos, is_core_own);
}

t_obj *ft_get_core_opponent(void)
{
	t_pos pos = {0, 0};
	return core_get_obj_filter_nearest(pos, is_core_opponent);
}

t_obj *ft_get_deposit_nearest(t_pos pos) { return core_get_obj_filter_nearest(pos, is_deposit); }
t_obj *ft_get_gems_nearest(t_pos pos) { return core_get_obj_filter_nearest(pos, is_gems); }
t_obj *ft_get_wall_nearest(t_pos pos) { return core_get_obj_filter_nearest(pos, is_wall); }
unsigned int ft_count_walls(void) { return core_get_objs_filter_count(is_wall); }
t_obj **ft_get_units_own(void) { return core_get_objs_filter(is_unit_own); }
t_obj **ft_get_units_opponent(void) { return core_get_objs_filter(is_unit_opponent); }
t_obj *ft_get_units_opponent_nearest(t_pos pos) { return core_get_obj_filter_nearest(pos,
is_unit_opponent); }

t_obj *ft_get_threat_nearest(t_pos pos, unsigned int max_dist)
{
	t_obj *opp = ft_get_units_opponent_nearest(pos);
	if (!opp)
		return NULL;
	if (ft_distance(pos, opp->pos) <= max_dist)
		return opp;
	return NULL;
```

```
}

bool ft_is_pos_near_armed_enemy(t_pos pos, unsigned int radius)
{
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
			o->s_unit.properties.damage_unit > 0 && o->hp > 0)
		{
			if (ft_distance(pos, o->pos) <= radius)
				return true;
		}
	}
	return false;
}

bool ft_is_enemy_adjacent_to_pos(t_pos pos)
{
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
			o->hp > 0 && ft_distance(pos, o->pos) <= 1)
			return true;
	}
	return false;
}

unsigned int ft_count_allies_near_pos(t_pos pos, unsigned int radius)
{
	unsigned int count = 0;
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		const t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id == game.my_team_id &&
			o->s_unit.properties.damage_unit > 0 && o->hp > 0 &&
			ft_distance(o->pos, pos) <= radius)
		{
			count++;
		}
	}
	return count;
```

```
}

t_obj *ft_get_best_adjacent_enemy(const t_obj *attacker)
{
	if (!attacker || attacker->s_unit.properties.damage_unit == 0)
		return NULL;

	t_obj **opps = ft_get_units_opponent();
	t_obj *best = NULL;
	long best_score = -999999;

	for (int i = 0; opps && opps[i]; i++)
	{
		t_obj *e = opps[i];
		if (!ft_is_adjacent(attacker->pos, e->pos))
			continue;

		unsigned long pending = get_pending_damage(e->id);
		if (pending >= e->hp)
			continue;

		unsigned long rem_hp = e->hp - pending;
		unsigned long raw_dmg = attacker->s_unit.properties.damage_unit;
		int red = e->s_unit.properties.damage_reduction_percent;
		unsigned long eff_dmg = (raw_dmg * (unsigned long)(100 - red)) / 100;
		if (eff_dmg == 0 && raw_dmg > 0)
			eff_dmg = 1;

		long score = 1000 - (long)(rem_hp * 30);

		if (eff_dmg >= rem_hp)
			score += 800;

		if (e->s_unit.properties.damage_unit > 0)
		{
			score += (long)(e->s_unit.properties.damage_unit * 60);
			if (e->s_unit.properties.base_action_cooldown <= 3)
				score += 250;
		}

		if (e->s_unit.gems > 0)
			score += 150;
```

```
                if (score > best_score)
                {
                        best_score = score;
                        best = e;
                }
        }
        free(opps);
        return best;
}

bool ft_is_core_threatened(t_obj *core, unsigned int radius)
{
        if (!core)
                return false;
        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                t_obj *o = game.objects[i];
                if (o->type == OBJ_UNIT && o->s_unit.team_id != game.my_team_id &&
                        (o->s_unit.properties.damage_unit > 0 ||
o->s_unit.properties.damage_core > 0))
                {
                        if (ft_distance(core->pos, o->pos) <= radius)
                                return true;
                }
        }
        return false;
}

unsigned int ft_count_active_resources(void)
{
        unsigned int count = 0;
        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                t_obj *o = game.objects[i];
                if (o->type == OBJ_DEPOSIT || o->type == OBJ_GEM_PILE)
                        count++;
        }
        return count;
}

unsigned int ft_count_safe_resources(t_pos core_pos)
{
        unsigned int count = 0;
```

```
	t_obj *opp_core = ft_get_core_opponent();
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		t_obj *o = game.objects[i];
		if (o->type != OBJ_DEPOSIT)
			continue;
		if (ft_is_pos_near_armed_enemy(o->pos, 3))
			continue;
		if (opp_core && game.elapsed_ticks <= 350 &&
			ft_distance(o->pos, core_pos) >= ft_distance(o->pos, opp_core->pos))
			continue;
		count++;
	}
	return count;
}

unsigned int ft_count_enemy_combat_units(void)
{
	unsigned int count = 0;
	t_obj **opps = ft_get_units_opponent();
	for (int i = 0; opps && opps[i]; i++)
	{
		if (opps[i]->s_unit.properties.damage_unit > 0 ||
opps[i]->s_unit.properties.damage_core > 0)
			count++;
	}
	free(opps);
	return count;
}

unsigned int ft_count_own_combat_units(void)
{
	unsigned int count = 0;
	t_obj **own = ft_get_units_own();
	for (int i = 0; own && own[i]; i++)
	{
		if (own[i]->s_unit.properties.damage_unit > 0)
			count++;
	}
	free(own);
	return count;
}
```

```
unsigned long ft_get_enemy_max_hp(void)
{
	unsigned long max_hp = 0;
	t_obj **opps = ft_get_units_opponent();
	for (int i = 0; opps && opps[i]; i++)
	{
		if (opps[i]->s_unit.properties.damage_unit > 0 && opps[i]->hp > max_hp)
			max_hp = opps[i]->hp;
	}
	free(opps);
	return max_hp;
}

unsigned int ft_count_enemy_speed_warriors(void)
{
	unsigned int count = 0;
	t_obj **opps = ft_get_units_opponent();
	for (int i = 0; opps && opps[i]; i++)
	{
		t_obj *e = opps[i];
		if (e->s_unit.properties.damage_unit > 0 &&
			(e->s_unit.properties.base_action_cooldown <= 3 || e->hp <= 6))
		{
			count++;
		}
	}
	free(opps);
	return count;
}

bool ft_is_enemy_engaged_by_ally(const t_obj *enemy, unsigned long exclude_unit_id)
{
	if (!enemy)
		return false;
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id == game.my_team_id && o->id !=
exclude_unit_id)
		{
			if (o->s_unit.properties.damage_unit > 0 && ft_distance(o->pos,
enemy->pos) <= 2)
				return true;
```

```
                }
        }
        return false;
}

// Verifies that an open cardinal tile exists at Manhattan distance 1 from both u and enemy
bool ft_has_open_melee_gap(const t_obj *u, const t_obj *enemy)
{
        if (!u || !enemy)
                return false;
        const int dx[4] = {1, 0, -1, 0};
        const int dy[4] = {0, 1, 0, -1};
        for (int d = 0; d < 4; d++)
        {
                int x = (int)u->pos.x + dx[d];
                int y = (int)u->pos.y + dy[d];
                if (x < 0 || y < 0 || x >= (int)game.grid_size || y >= (int)game.grid_size)
                        continue;
                t_pos p = {(unsigned short)x, (unsigned short)y};
                if (ft_distance(p, enemy->pos) != 1)
                        continue;
                const t_obj *o = core_get_obj_from_pos(p);
                if (!o || (o->type == OBJ_GEM_PILE &&
                        o->s_deposit_gems_pile.gems <= u->s_unit.properties.max_gems -
u->s_unit.gems))
                        return true;
        }
        return false;
}

bool ft_should_hold_for_first_strike(const t_obj *u, const t_obj *target)
{
        if (!u || !target || target->type != OBJ_UNIT)
                return false;
        if (ft_distance(u->pos, target->pos) != 2)
                return false;
        if (target->s_unit.properties.damage_unit == 0)
                return false;
        if (ft_is_enemy_engaged_by_ally(target, u->id))
                return false;
        if (!ft_has_open_melee_gap(u, target))
                return false;
        if (u->s_unit.action_cooldown <= -4)
```

```
                return false;

        long enemy_cd = target->s_unit.action_cooldown;
        long my_base_cd = (long)u->s_unit.properties.base_action_cooldown;
        if (enemy_cd >= my_base_cd || enemy_cd <= -5)
                return false;

        return true;
}

static unsigned long estimate_team_power(unsigned long team_id, unsigned int
*combat_count)
{
        unsigned long total_power = 0;
        unsigned int count = 0;

        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                t_obj *o = game.objects[i];
                if (o->type != OBJ_UNIT || o->s_unit.team_id != team_id || o->hp == 0)
                        continue;

                unsigned long dmg_u = o->s_unit.properties.damage_unit;
                unsigned long dmg_c = o->s_unit.properties.damage_core;
                if (dmg_u == 0 && dmg_c == 0)
                        continue;

                count++;
                unsigned long cd = (o->s_unit.properties.base_action_cooldown > 0)
                        ? (unsigned long)o->s_unit.properties.base_action_cooldown : 1;
                unsigned long armor_mult = 100 + (unsigned
long)o->s_unit.properties.damage_reduction_percent;
                unsigned long eff_hp = (o->hp * armor_mult) / 100;
                unsigned long dps_score = ((dmg_u * 3 + dmg_c) * 6) / cd;

                total_power += eff_hp * 10 + dps_score * 15;
        }

        if (combat_count)
                *combat_count = count;
        return total_power;
}
```

```
t_game_phase ft_get_game_phase(t_obj *core, t_obj *opp_core)
{
	if (!opp_core || opp_core->hp == 0)
		return PHASE_3_ALL_OUT_ASSAULT;

	unsigned int active_res = ft_count_active_resources();
	unsigned int my_combat_units = 0;
	unsigned int opp_combat_units = 0;
	unsigned long my_power = estimate_team_power(game.my_team_id,
&my_combat_units);
	unsigned long opp_power = estimate_team_power(opp_core->s_core.team_id,
&opp_combat_units);

	if (opp_combat_units == 0 && my_combat_units >= 1)
		return PHASE_3_ALL_OUT_ASSAULT;

	if (my_combat_units >= MIN_ASSAULT_ARMY_SIZE &&
		(my_power * 100 >= opp_power * ASSAULT_POWER_RATIO_PCT))
	{
		return PHASE_3_ALL_OUT_ASSAULT;
	}

	if (active_res > PHASE2_RESOURCE_CUTOFF)
		return PHASE_1_AGGRESSIVE_ECONOMY;

	if (core && core->s_core.gems >= COST_HUNTER_TANK && my_combat_units < 10)
		return PHASE_2_TACTICAL_DEFENSE;

	if (my_power >= opp_power && my_combat_units >= 2)
		return PHASE_3_ALL_OUT_ASSAULT;

	return PHASE_2_TACTICAL_DEFENSE;
}

static bool is_claimed_by_other_miner(t_pos res_pos, unsigned long my_id)
{
	for (int i = 0; game.objects && game.objects[i]; i++)
	{
		t_obj *o = game.objects[i];
		if (o->type == OBJ_UNIT && o->s_unit.team_id == game.my_team_id && o->id !=
my_id)
		{
			if (o->s_unit.name && strcmp(o->s_unit.name, "Miner") == 0)
```

```
                        {
                                if (ft_distance(o->pos, res_pos) <= 2)
                                        return true;
                        }
                }
        }
        return false;
}

t_obj *ft_get_resource_for_miner(t_pos unit_pos, t_pos core_pos, unsigned long unit_id)
{
        t_obj *core = ft_get_core_own();
        t_obj *opp_core = ft_get_core_opponent();
        t_obj *best = NULL;
        long best_score = LONG_MIN;
        unsigned int n_carriers = core_get_units_byName_count("Carrier");

        for (int i = 0; game.objects && game.objects[i]; i++)
        {
                t_obj *obj = game.objects[i];
                if (obj->type != OBJ_DEPOSIT && obj->type != OBJ_GEM_PILE)
                        continue;
                if (obj->s_deposit_gems_pile.gems == 0)
                        continue;

                // If Carrier can reach a Gem Pile, leave it for the Carrier!
                if (obj->type == OBJ_GEM_PILE && n_carriers > 0 &&
ft_carrier_can_reach_pile(obj, core))
                        continue;

                if (ft_is_pos_near_armed_enemy(obj->pos, 3))
                        continue;

                if (is_claimed_by_other_miner(obj->pos, unit_id))
                        continue;

                unsigned int dist_u = ft_distance(unit_pos, obj->pos);
                unsigned int dist_c = ft_distance(core_pos, obj->pos);
                long score = 1000 - (long)(dist_u * 25) - (long)(dist_c * 12);

                if (opp_core && ft_distance(opp_core->pos, obj->pos) <= dist_c)
                        score -= 250;
```

```
                if (obj->type == OBJ_DEPOSIT)
                {
                        score -= 180 + (long)(obj->hp * 3);
                        // With an active Carrier, strongly prioritize cracking Deposits over locked
piles
                        if (n_carriers > 0)
                                score += 10000;
                }

                if (score > best_score)
                {
                        best_score = score;
                        best = obj;
                }
        }
        return best;
}
```

8. src/main.c (or main.c)

In main.c, two adjustments align your tick execution with Draft: nav_yield_for_workers is called immediately after handle_carrier and handle_miners and before handle_combat so that a combat unit sitting in a worker corridor yields its tile before handle_combat moves it, and debug in core_startGame is set to false to avoid console and socket logging overhead during tournament matches[5].

C

```
#include "bot.h"

void ft_on_tick(unsigned long tick)
{
        (void)tick;
        reservations_init_tick();

        t_obj *core = ft_get_core_own();
        t_obj *opp_core = ft_get_core_opponent();
        t_obj **units = ft_get_units_own();

        // 1. Run Carrier & Miner logistics first
        handle_carrier(core, units);
        handle_miners(core, units);
```

```
        // 2. Yield worker corridors before combat units take standard moves
        nav_yield_for_workers(core, opp_core, units);

        // 3. Military Operations: Hunters, Defenders, and Finisher Strikers
        handle_combat(core, opp_core, units);

        // 4. Spawner decision tree (uses confirmed + same-tick transferred Core gems)
        handle_spawner(core);

        if (units)
                free(units);
}

int main(int argc, char **argv)
{
        return core_startGame("No_Plan_Just_Prayers",
                                                argc,
                                                argv,
                                                ft_on_tick,
                                                false);
}
```

4. Your 12-Hour Execution Plan to Win the Bracket

With 12 hours remaining before the tournament bracket closes, follow this five-step engineering checklist to validate, tune, and deploy your upgraded bot[5]:

1. **Hours 0–1 (Drop-In Patch & Local Build Verification):** Replace the code in bot.h, navigation.c, carrier.c, miners.c, combat.c, spawner.c, getters.c, and main.c with the upgraded modules above. Run make re inside my-core-bot/ and confirm that the ./bot binary builds cleanly with zero compiler or linker warnings[5].
2. **Hours 1–3 (Head-to-Head Scrimmages Against Draft and Built-in Bots):** Run 20 local matches across varied map seeds against Draft (Monir's bot), gridmaster, and freddys-secret-bot[5]. In the match visualizer, verify two specific mechanical behaviors: first, when your 187-gem Miner breaks a Deposit, confirm that it leaves the 150-gem pile on the ground and walks to the next Deposit while your 232-gem Carrier scoops the entire 150-gem pile in one action[9]; second, when your Hunter pack clears the last enemy unit, confirm that they take up flanking guard posts at distance 2–4 around the enemy Core while leaving an open corridor for your Striker to walk up to distance 1 and destroy the Core[8].
3. **Hours 3–5 (Live Config Sanity Check):** Run ./scripts/check_update_configs.sh (or inspect configs/) to verify whether the tournament organizers pushed any mid-event

balance updates to component costs, deposit HP, or cooldown steps[5]. If any component cost changed, update the #define COST_* macros in bot.h to match the live server config[5].

4. **Hours 5–9 (Online Queue Testing & Threshold Calibration):** Push this verified version to the main branch of your official repository—remembering that the tournament runner only builds main and executes cd /shared-data/repo/my-core-bot && make && ./bot <rndId>—and queue live matchmaking games on the event website[5]. Watch your live queue replays to see whether opponents are rushing early 1-combat units or turtling behind walls; if opponents rush early on open maps, increase the ft_is_core_threatened radius check in spawner.c from 6 to 8[3].
5. **Hours 9–12 (Feature Freeze & Container Compliance Audit):** Freeze your codebase at least 90 minutes before the final bracket[5]. Confirm that debug is set to false in main.c, all dynamically allocated object arrays (units, opps) are freed every tick, the my-core-bot directory structure and Makefile remain unmodified, and your final tested commit is pushed to main[5].

Works cited

1. navigation.c
2. carrier.c
3. spawner.c
4. miners.c
5. 42 CORE Bot Architecture Strategy, uploaded:42 CORE Bot Architecture Strategy
6. main.c
7. bot.h
8. sword.c
9. miner.c
10. combat.c
11. unknown_url
12. flank_breaker.c
13. collector.c
14. core_lib.h
15. getters.c
16. CORE-Game-World-Championship-No_Plan_Just_Prayers.zip